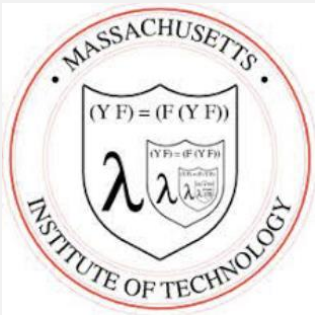


Programação Funcional

Unidade 1 – Paradigma Funcional de Programação



Prof. Aparecido V. de Freitas
Doutor em Engenharia
da Computação pela EPUSP
aparecido.freitas@maua.br
aparecidovfreitas@gmail.com

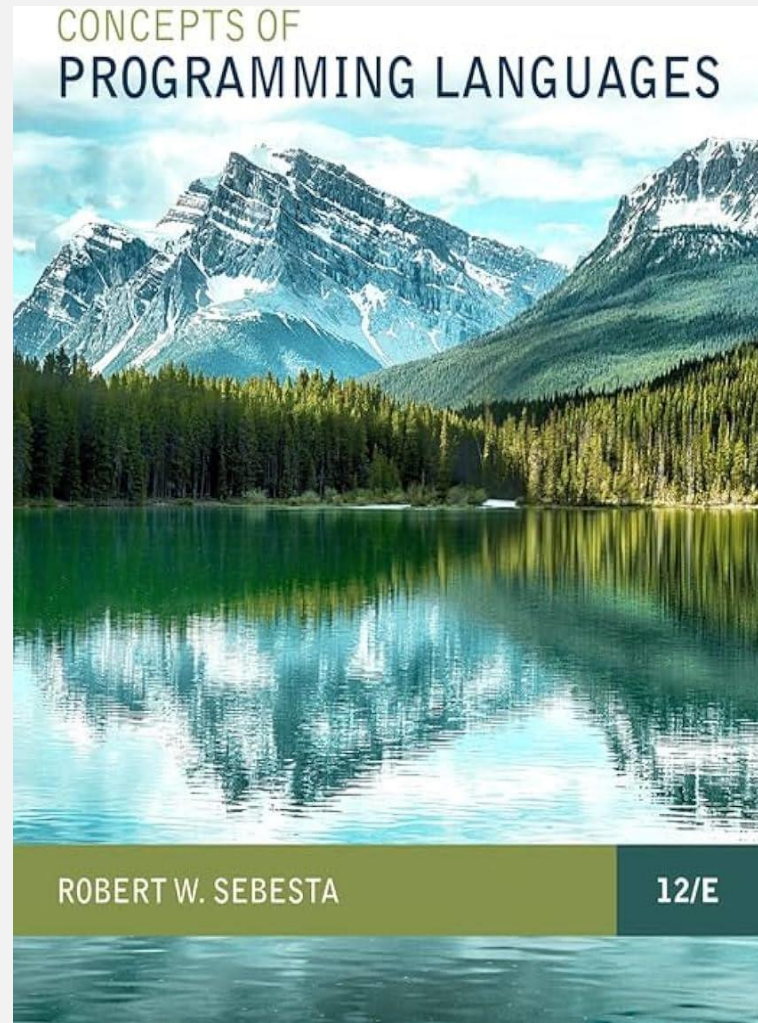
Prof. Dr. Aparecido V. de Freitas



- ◆ **Doutor** em Engenharia da Computação pela **EPUSP** – Escola Politécnica da **USP**
- ◆ **Mestre** em Engenharia da Computação pela **EPUSP** – Escola Politécnica da **USP**
- ◆ Especialização em Engenharia de Software pela EPUSP – Escola Politécnica da **USP**
- ◆ **Engenheiro Civil** pela Escola de Engenharia **Mauá**
- ◆ Bacharel em **Matemática** pela **Fundação Santo André**
- ◆ Atuou durante 15 anos como Supervisor de **TI** na área de TI da **Volkswagen** do Brasil
- ◆ Especialista na plataforma **IBM i** (desde 1993)
- ◆ Experiência na plataforma **IBM Mainframe** (15 anos)
- ◆ Professor da **USCS** desde a primeira turma do curso de Ciência da Computação (1989)
- ◆ Professor convidado dos cursos de **Computação** da Escola de Engenharia **Mauá**
- ◆ Gestor dos cursos de Computação da **USCS** (2000 a 2013) e (2020 a 2021)
- ◆ Ex-Professor do curso de Ciência da Computação da Universidade **Metodista**
- ◆ Ex-Professor do Curso de Matemática – Ênfase Software – **Fundação Santo André**
- ◆ Consultor e Instrutor em empresas de TI – Qualitsys Consultoria de Informática Ltda
- ◆ Certificação Internacional em Engenharia de Software – **ISTQB** – **CTFL**, **CTFL-AT**, **CPRE**
- ◆ Especialista – **CEE** - Conselho Estadual de Educação - SP

Bibliografia

■ R. Sebesta – Concepts Of Programming Languages



O que é Programação Funcional ?





Programação
Funcional nos faz
lembrar em Função ?

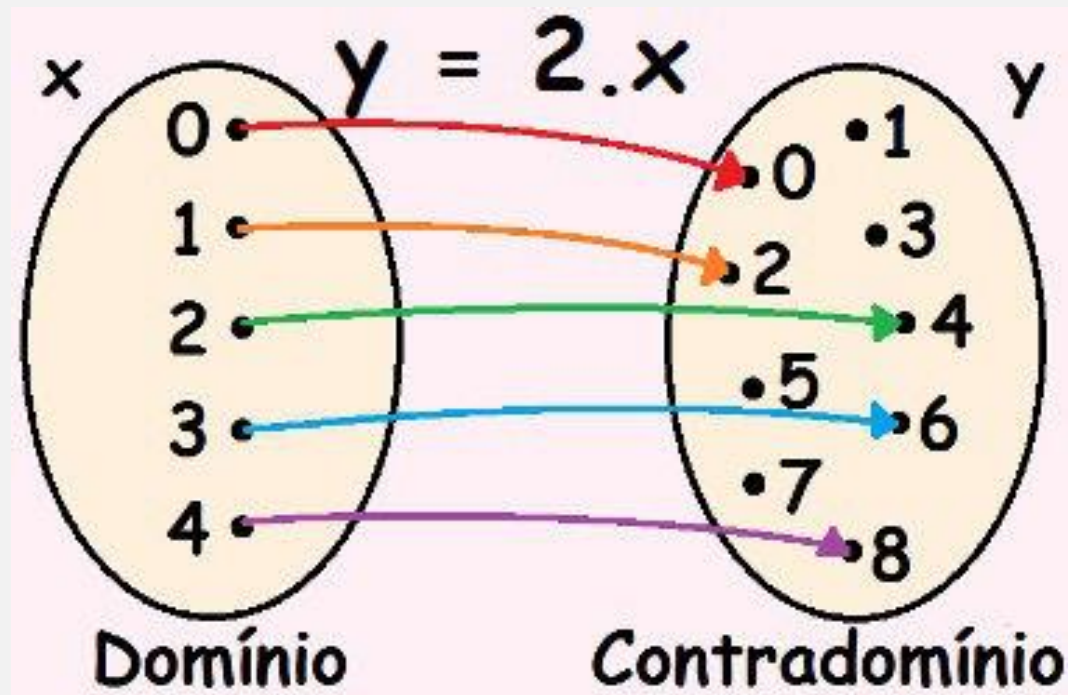
Mas, o que é Função ?





... lembrando das aulas de
Cálculo Diferencial e Integral

Função é uma **regra** que associa cada elemento de um **conjunto** a um único elemento de outro **conjunto**.

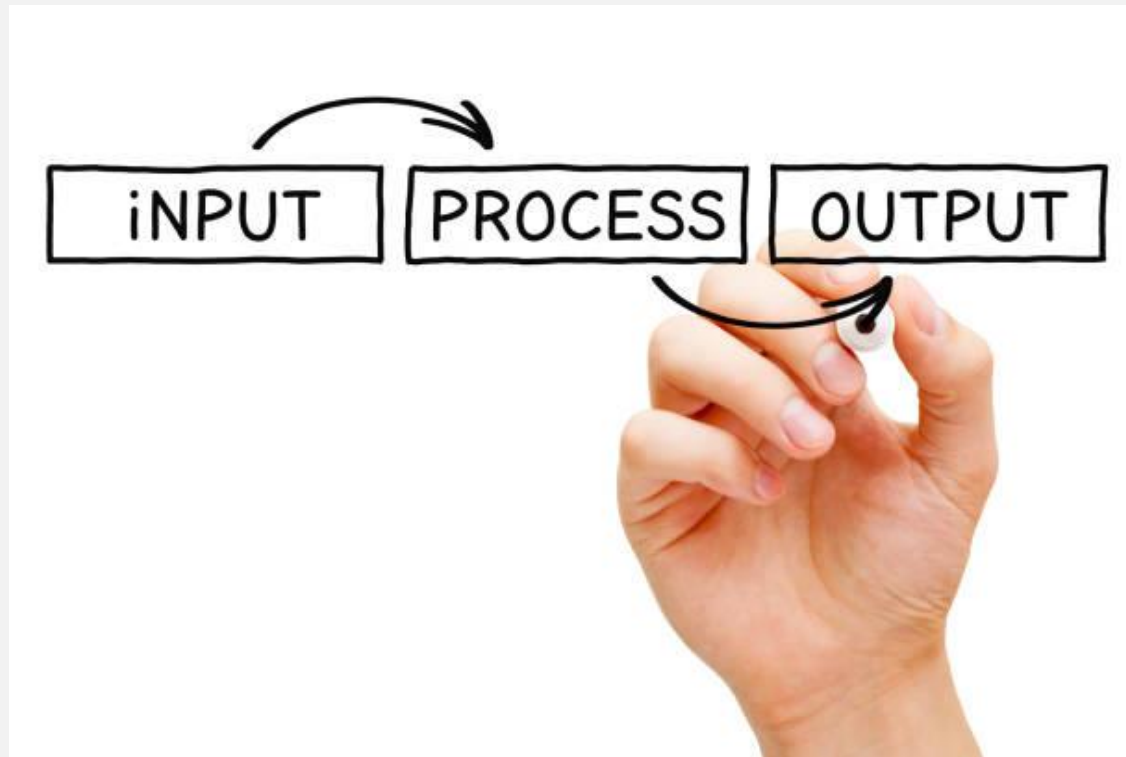


Existe algum relacionamento entre Função e Programação ?



Função e Programação

Funções realizam **mapeamento** de dados e a programação usa esse conceito para **transformar** dados em informações.



Uma primeira definição

Programação funcional é um **paradigma** de programação que usa **funções**.



Lambda

$f A : (A, A) \rightarrow A$



$\text{map } B : (B, B) \rightarrow B$



$\text{flatMap } A : (A, A) \rightarrow A$



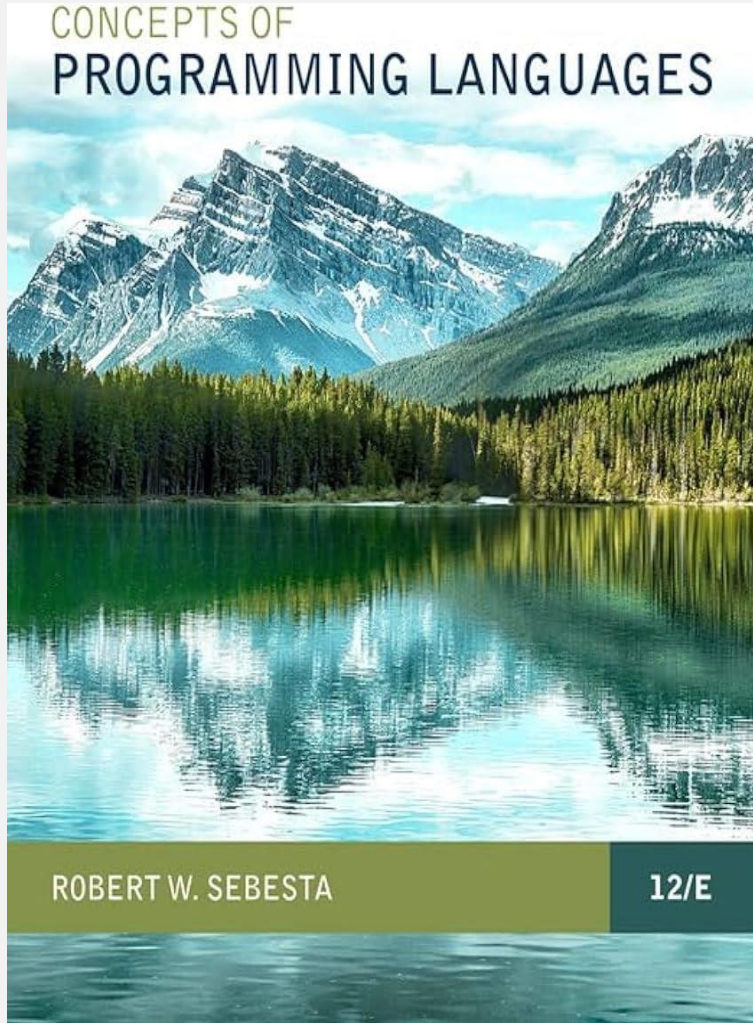
Mas, o que é paradigma ?

Paradigma de Programação



Paradigma de programação é um **modelo** ou forma de pensar e escrever programas.

Paradigma de Programação



❖ Imperativo

❖ OOP

❖ Funcional

❖ Lógico

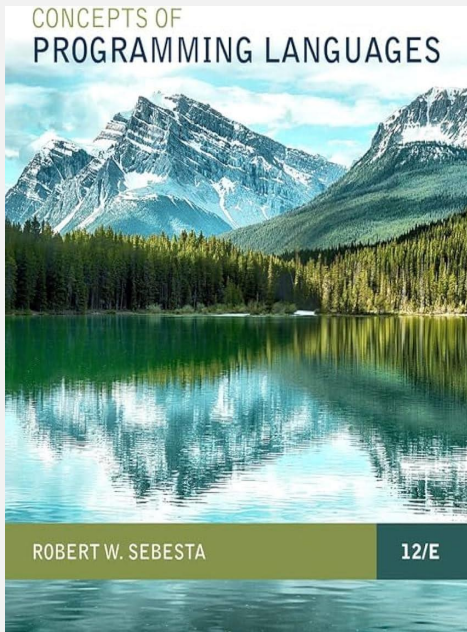


Mas, C é imperativo e usa funções !
Java é OOP e usa funções !



O que então caracteriza uma Linguagem Puramente Funcional ?

Linguagens Puramente Funcionais



“Uma linguagem **puramente** funcional é aquela que impõe o uso exclusivo de **funções matemáticas** e dados **imutáveis**, **evitando** a reatribuição de variáveis e, conseqüentemente, a alteração do estado do programa.”



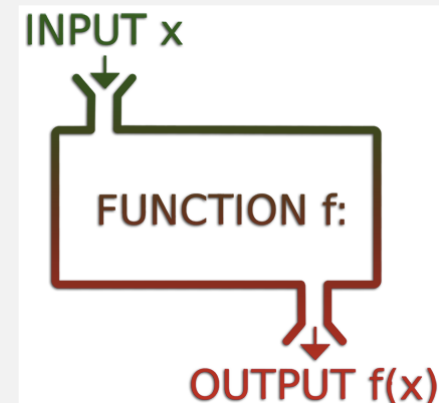
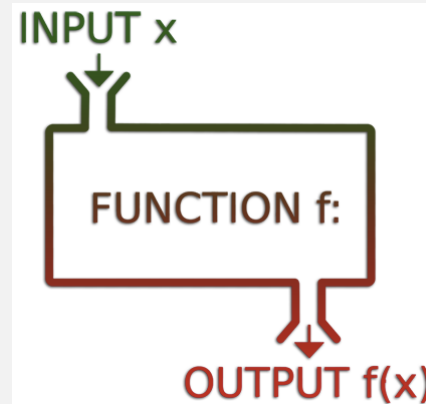
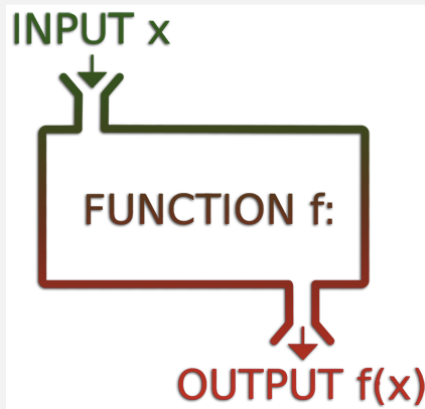
Em que diferem as Linguagens Imperativas das Puramente Funcionais ?

Programação Imperativa

- ❖ Baseada em **estados** de Memória;
- ❖ Programa tem variáveis que ocupam posições de memória;
- ❖ A execução do programa consiste em modificar esses valores (**estado**);
- ❖ C, Pascal, Fortran, Cobol

Programação Funcional

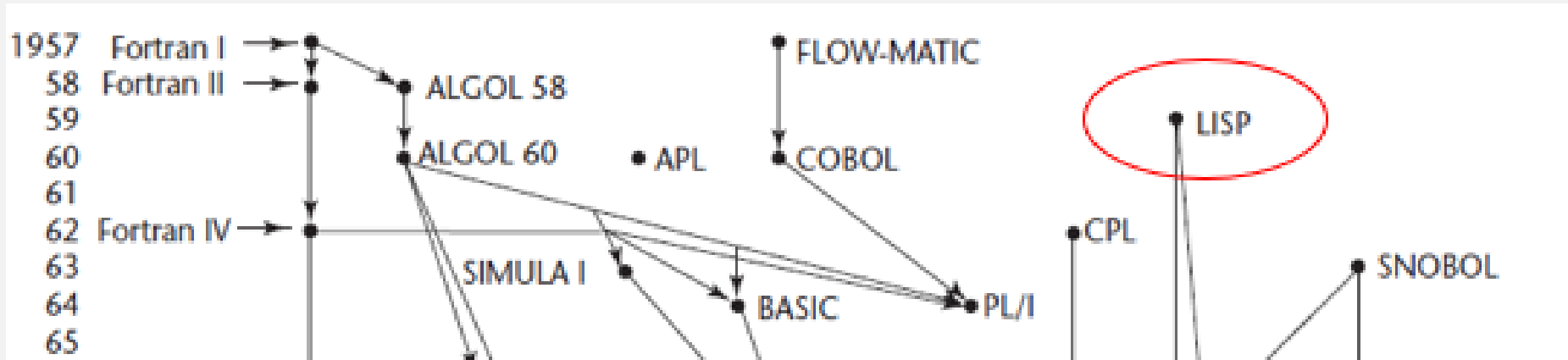
- ❖ Transformação de Dados
- ❖ Imutabilidade
- ❖ Transparência Referencial
- ❖ Funções de Alta Ordem



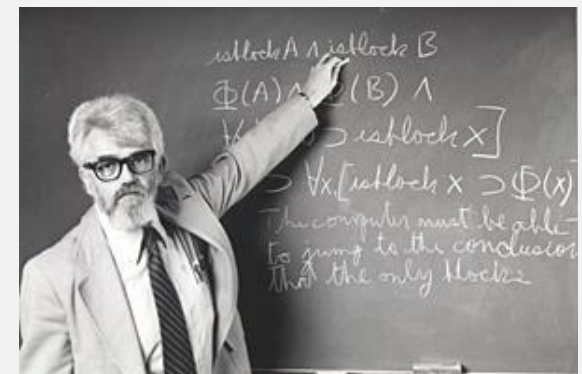


Linguagens Funcionais são recentes ?

Lisp



- ❖ Concebida por **John McCarthy** em **1959**, no **MIT**;
- ❖ Focada no uso exclusivo de funções matemáticas como estrutura de dados;
- ❖ Tem como base formal o **Cálculo Lambda** de Alonzo Church (**1941**);



MIT



- ❖ Papel central no desenvolvimento prático e teórico da **Programação Funcional**;
- ❖ Desenvolvimento da Linguagem **Lisp** e **Scheme**.



Como aprender Programação Funcional ?

<http://www.newlisp.org>

newlisp.org/index.cgi?Documentation

Drive  GDrive  R  T  g   Ms  Prime  aws  Oracle  ibm  Foto  libgen  FStack  Y  F  ML



puts the fun

back in Lisp

[Home](#) · [About](#) · [Downloads](#) · [Docs](#) · [Modules](#) · [Links](#) · [Tips&Tricks](#)

Documentation

Main documentation and man pages

- newLISP Manual and Reference in [HTML](#) format, no frames.
newLISP Manual and Reference in [HTML](#) format, with index frame.

<http://www.newlisp.org>

CONCEPTION OF ADAPTIVE PROGRAMMING LANGUAGES

Aparecido Valdemir de Freitas^{1,2}, João José Neto¹

¹ Escola Politécnica da Universidade de São Paulo

Depto. de Engenharia de Computação e Sistemas Digitais

Av. Prof. Luciano Gualberto, trav. 3, N° 158. Cidade Universitária - São Paulo – Brasil

² Universidade Municipal de São Caetano do Sul

Instituto Municipal de Ensino Superior de São Caetano do Sul

Av. Goiás N° 3400 – Vila Barcelona – São Caetano do Sul – CEP 09550-051 - São Paulo – Brasil

avfreitas@imes.edu.br and joao.jose@poli.usp.br

ABSTRACT

Adaptive devices show the characteristic of dynamically change themselves in response to input stimuli with no interference of external agents. Occasional changes in behaviour are immediately detected by the devices, which right away react spontaneously to them. Chronologically

2. Adaptive Automata

Adaptive automata are special adaptive devices whose underlying formalism is the structured pushdown automaton. Their operation corresponds to a sequence of successive evolutions performed by an initial structured pushdown automaton processed by adaptive actions. To

<http://www.newlisp.org>

Adaptive device with underlying mechanism defined by a programming language

APARECIDO VALDEMIR DE FREITAS^{1,2}, JOÃO JOSÉ NETO¹

¹ Escola Politécnica da Universidade de São Paulo

Depto. de Engenharia de Computação e Sistemas Digitais

Av. Prof. Luciano Gualberto, trav. 3, N° 158. Cidade Universitária

São Paulo – Brasil

² Universidade Municipal de São Caetano do Sul

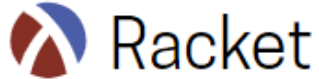
Instituto Municipal de Ensino Superior de São Caetano do Sul

Av. Goiás N° 3400 – Vila Barcelona – São Caetano do Sul – CEP 09550-051

São Paulo – Brasil

avfreitas@imes.edu.br and joao.jose@poli.usp.br

Abstract: - An adaptive device is made up of an underlying mechanism, for instance, an automaton, a grammar, a decision tree, etc., to which is added an adaptive mechanism, responsible for allowing a dynamic modification in the structure of the underlying mechanism. This article aims to investigate if a programming

[DONATE](#)[DOCS](#)[PACKAGES](#)[DOWNLOAD](#)

Racket version 7.8 is available.

tenth RacketCon will be held online in October.

Racket, the Programming Language

Mature

Jet Fueled

Extensible

Robust

Polished

Vibrant Community



```
#lang racket/gui
```

```
(define my-language 'English)
```

```
(define translations  
  #hash([English . "Hello world"]  
        [French . "Bonjour le monde"]  
        [German . "Hallo Welt"]  
        [Greek . "Γειά σου, κόσμος"]  
        [Portuguese . "Olá mundo"]  
        [Spanish . "Hola mundo"]  
        [Thai . "สวัสดีชาวโลก"]))
```

```
(define my-hello-world  
  (hash-ref translations my-language  
             "hello world"))
```

```
(message-box "" my-hello-world)
```

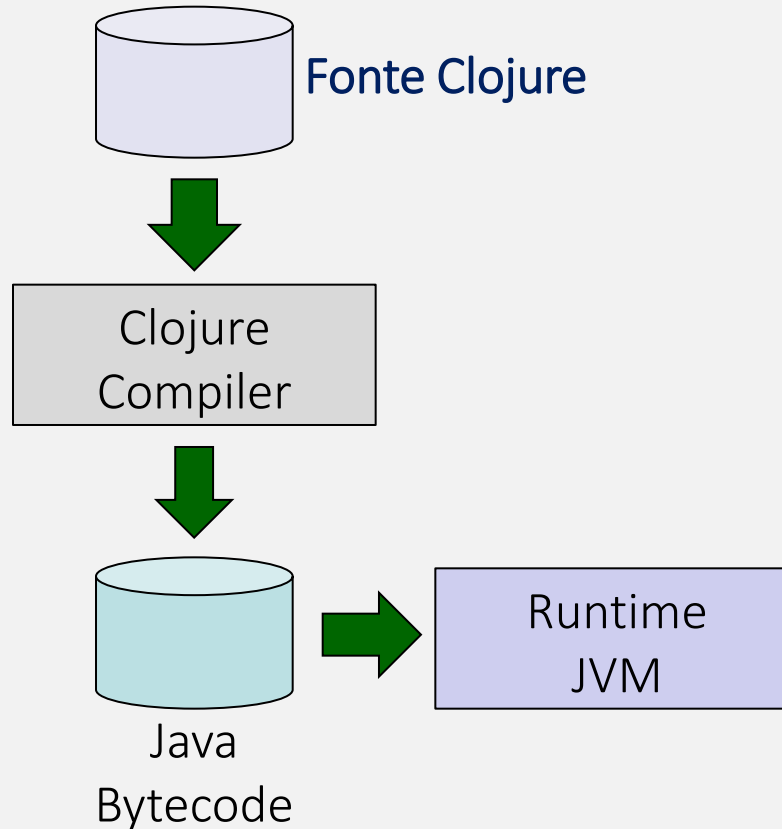
Clojure



- ❖ Criada por **Rich Hickey** em **2010** com o objetivo de combinar:
 - ✓ o poder expressivo das linguagens **Lisp**;
 - ✓ os princípios da **programação funcional**;
 - ✓ a robustez e o ecossistema da **Java Virtual Machine (JVM)**.

Clojure

- ❖ Compilador implementado em **Java**;
- ❖ Roda na **JVM** (Java Virtual Machine);



Clojure roda na JVM

- ❖ A Java Virtual Machine (JVM) é uma das plataformas de execução **mais testadas, seguras e estáveis** da história da computação;
- ❖ Usada há mais de **25 anos** em grandes organizações, sistemas críticos, servidores de grande escala.



Clojure – Características

- ❖ Dialeto **Lisp**;
- ❖ Puramente Funcional;
- ❖ Imutabilidade por padrão;
- ❖ Excelente suporte à concorrência;
- ❖ Interoperabilidade total com **Java**;
- ❖ Forte ligação com a **JVM**.





Clojure é usada no mercado ?

Clojure e o mercado

- ❖ Não é padrão de mercado;
- ❖ Porém, é largamente empregada no Nubank.

<https://nubank.com.br/sobre-o-nu>

127 milhões

de clientes no Brasil, México e Colômbia.

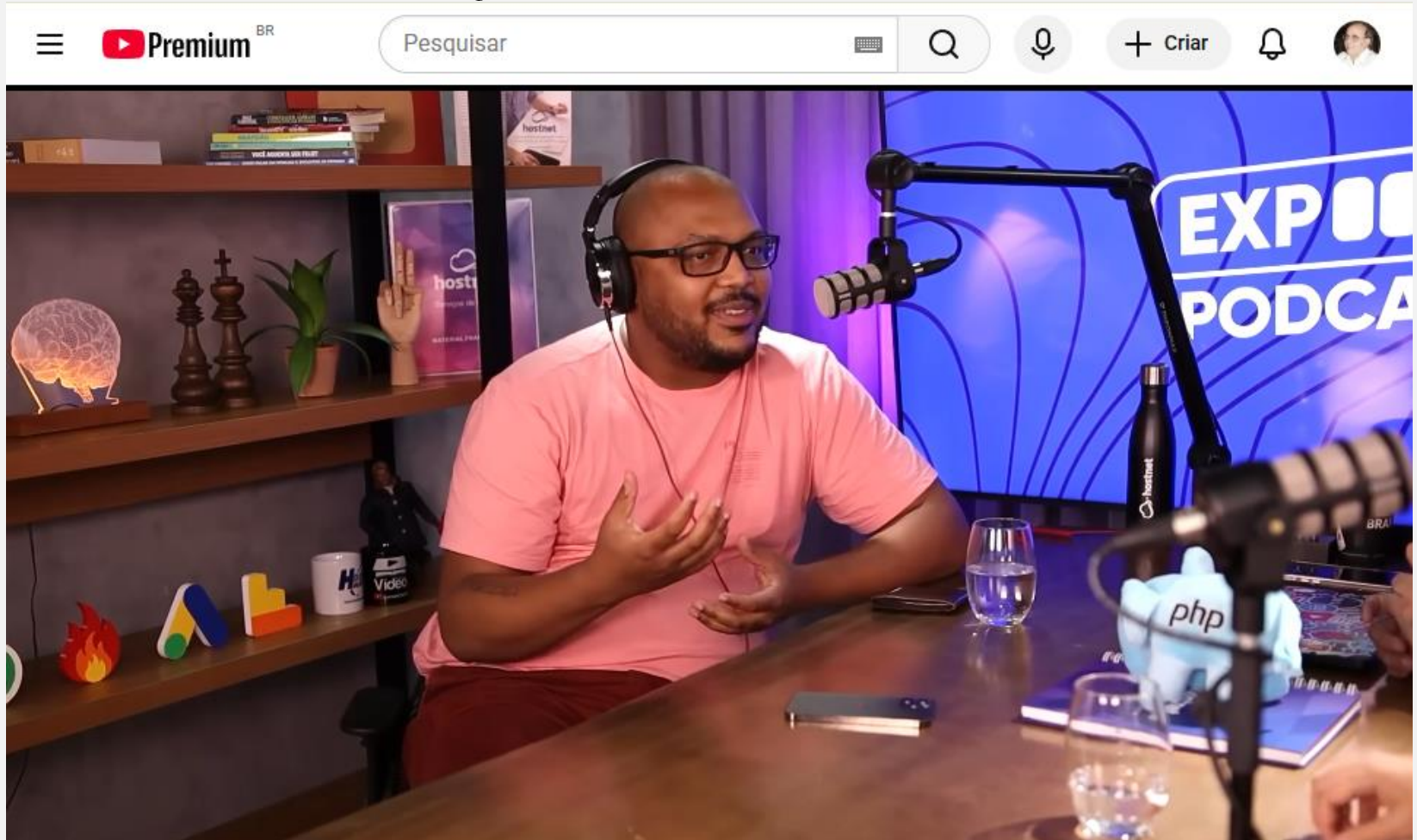


Clojure e Nubank

- ❖ A área de TI do **Nubank** tem por volta de 1500 a 2000 profissionais;
- ❖ **Clojure** é a linguagem **estratégica** do Nubank;
- ❖ Usada no **core** do backend;
- ❖ Muito valorizada em **processos seletivos**.



Clojure e Nubank



O NUBANK tem uma LINGUAGEM DE PROGRAMAÇÃO



CursoemVideo Cortes

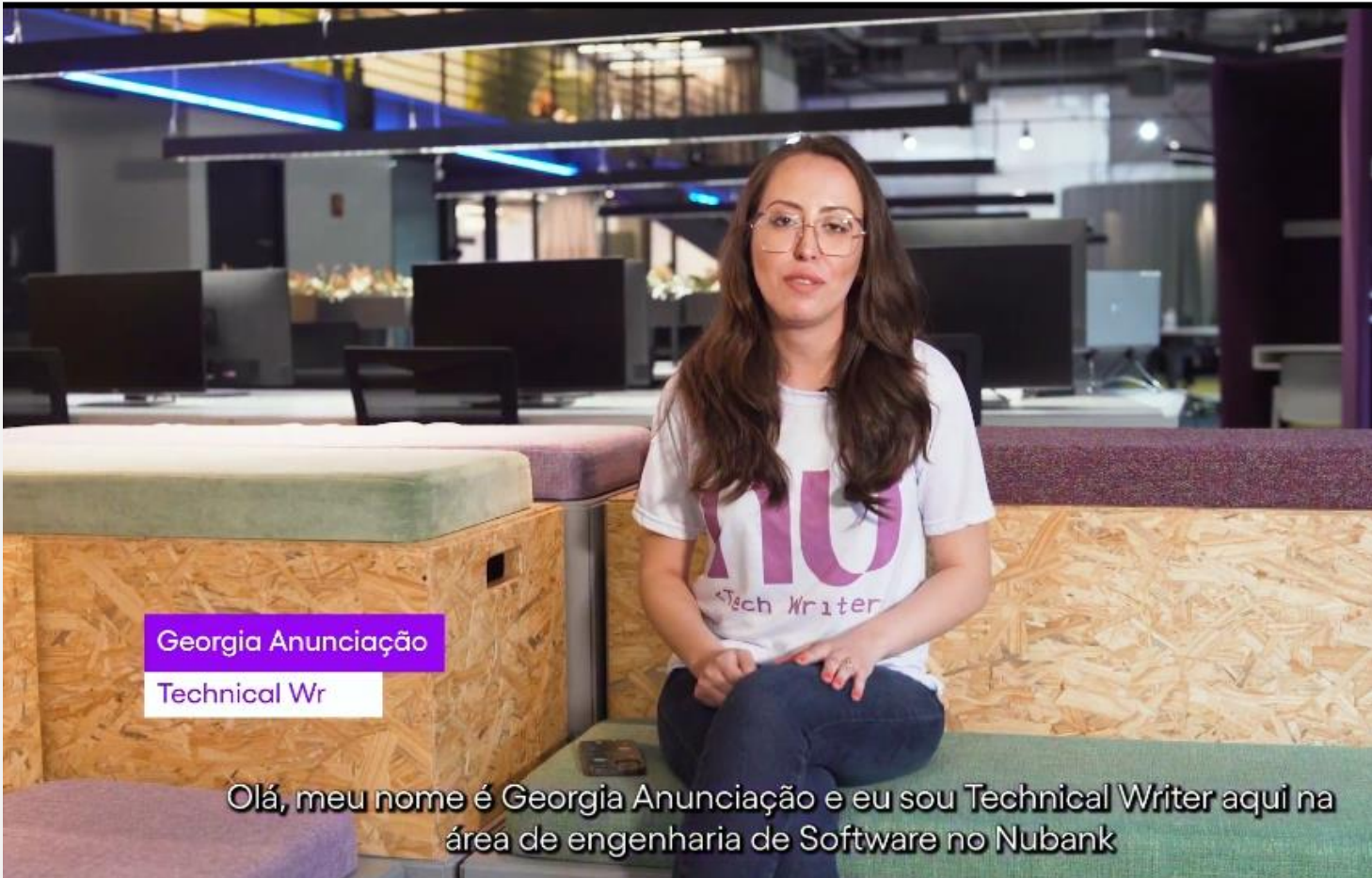
12,3 mil inscritos



Programação Funcional – Unidade 1 – Paradigma Funcional

Clojure e Nubank ?

Menu Premium^{BR} Pesquisar Q Microphone + Criar



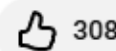
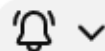
Georgia Anunciação
Technical Wr

Olá, meu nome é Georgia Anunciação e eu sou Technical Writer aqui na área de engenharia de Software no Nubank

De onde veio o Clojure? Qual a relação com o Nubank?



Building Nubank
20,4 mil inscritos



308



Clojure e Nubank

<https://building.nubank.com/functional-programming-with-clojure/>

Programação funcional com Clojure: por que e como o Nubank usa e escala tão bem?

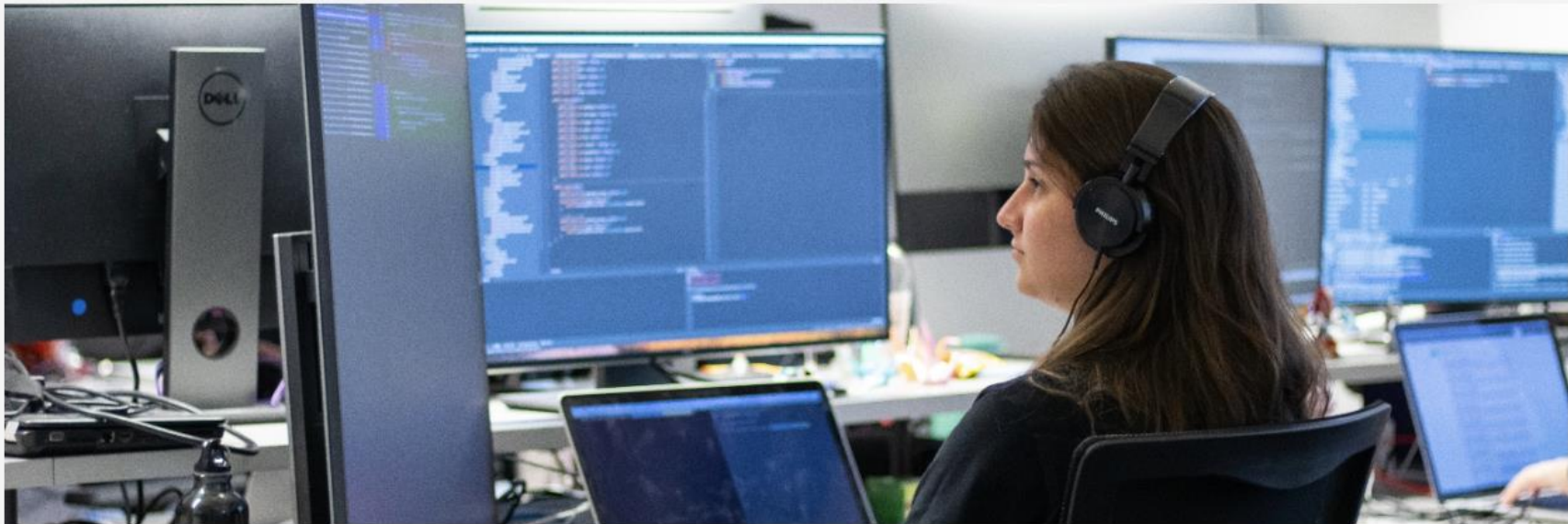
28 Mar, 23

Today, **Nubank's microservices architecture** has around 1000 microservices written in Clojure. Moreover, all of the microservice have the same folder structure based in our



Clojure e Nubank

Microsserviços no Nubank, uma visão geral



Microsserviços são o estilo arquitetônico predominante da atualidade. Para o nosso propósito, não vale a pena descrever aqui o que significa e suas vantagens, pois há muito conteúdo excelente disponível por aí. Em vez disso, vamos ser mais específicos e

MAIS LIDOS

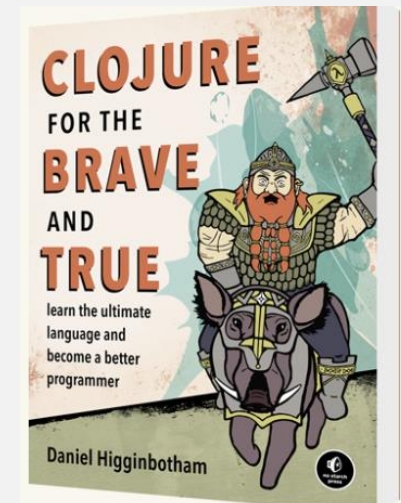
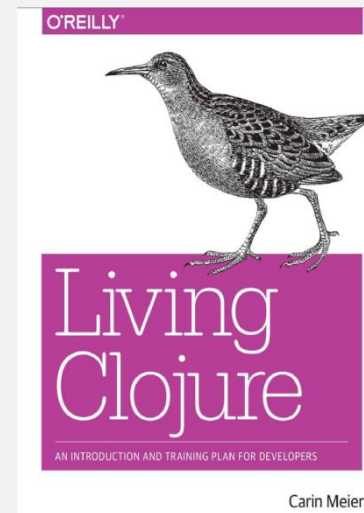
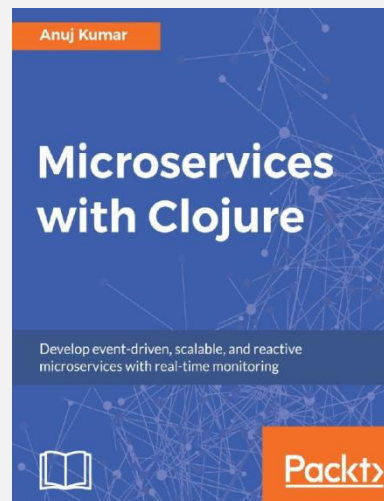
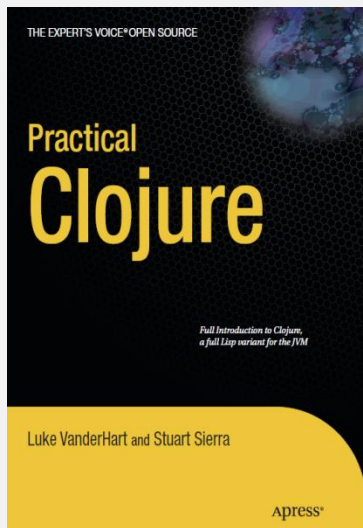
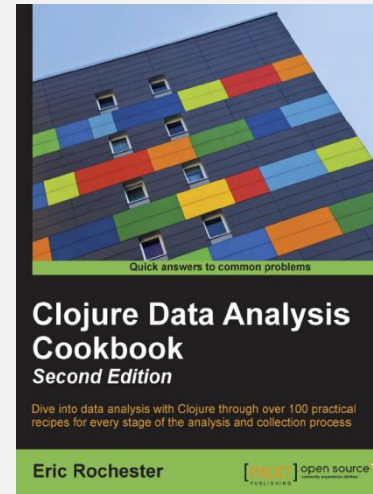
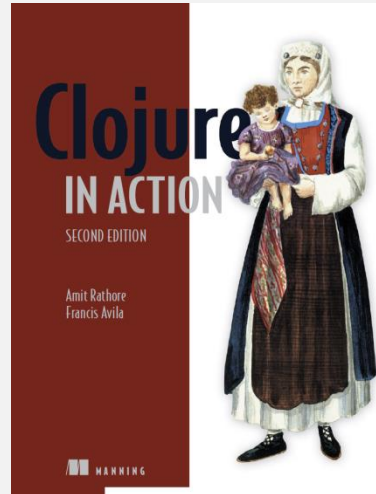
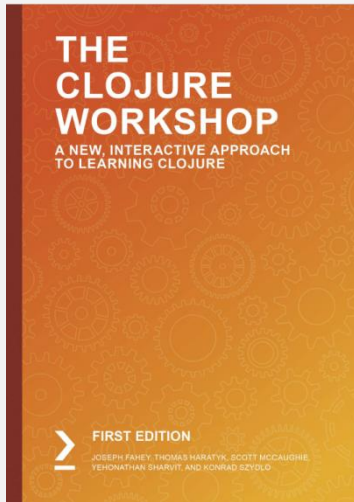
LIFE AT NU

Conheça a sede do Nubank em

Como aprender Clojure ?



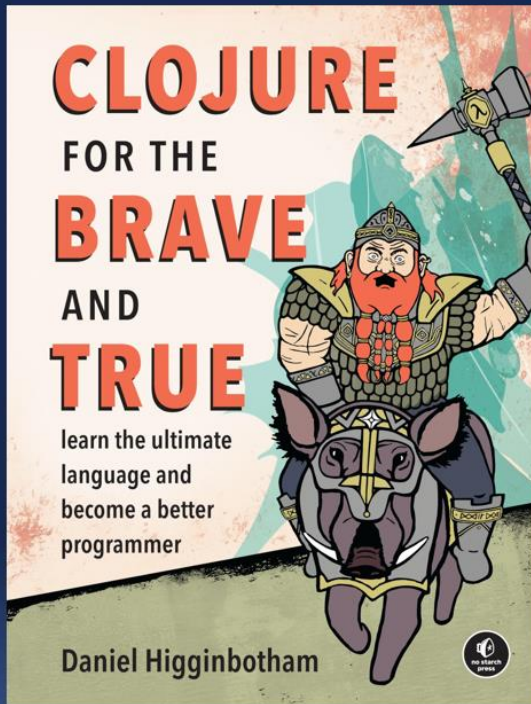
Como aprender Clojure ?



Como aprender Clojure ?

QualitySys.com IMT – Instituto Mauá de Tecnologia www.qualitysys.com

Programação Funcional



Prof. Aparecido V. de Freitas
Doutor em Engenharia
da Computação pela EPUVSP
aparecido.freitas@maua.br
aparecidovfreitas@gmail.com

Como aprender Clojure ?



Clojure: Introdução à Programação Funcional

Aprenda os fundamentos do paradigma Funcional...

Marcio Frayze David

★ 4,8

182 classificações

11 horas no total

53 aulas



Clojure: The Complete Beginner's Guide

Learn Clojure, a fun new functional programming...

Catalin Stefan

★ 4,8

998 classificações

4 horas no total

40 aulas

Grupo – Clojure Brasil





Como instalar Clojure ?

Instruções de Instalação

Local build

Download and build Clojure from source (requires Git, Java, and Maven):

```
git clone https://github.com/clojure/clojure.git  
cd clojure  
mvn -Plocal -Dmaven.test.skip=true package
```

Then start the REPL with the local jar:

```
java -jar clojure.jar
```

Leiningen – www.leiningen.org

- ✓ Foco na automação do projeto e configuração declarativa, permite que se foque na criação do código;
- ✓ Baseada em **Maven** !



Leiningen

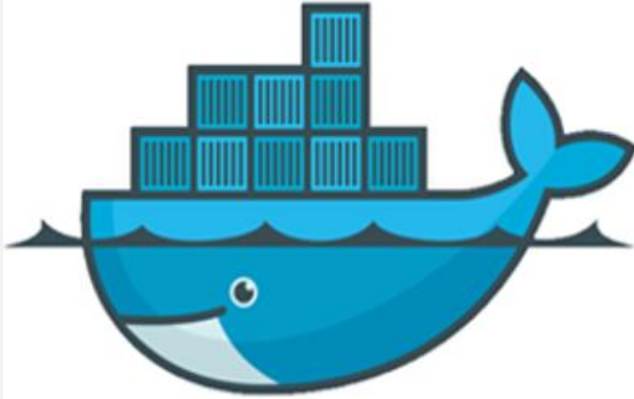
for automating Clojure projects without setting your hair on fire

Maven

Instalação com Docker

1. Criar o **Dockerfile**;
2. Build da imagem; (**Docker hub**)
3. Processar Clojure (REPL)





 Dockerfile ✕

D: > clojure >  Dockerfile > ...

```
1 FROM clojure:tools-deps
2 WORKDIR /app
```

```
Windows PowerShell
PS D:\clojure> ls

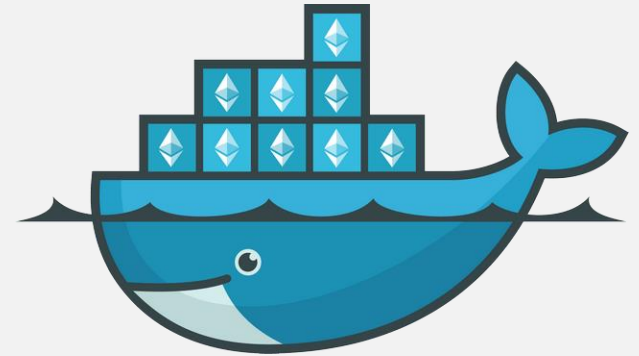
Diretório: D:\clojure

Mode                LastWriteTime         Length Name
----                -
-a-----         05/02/2026   11:02         39 Dockerfile

PS D:\clojure> |
```

`docker build -t clj-repl .`

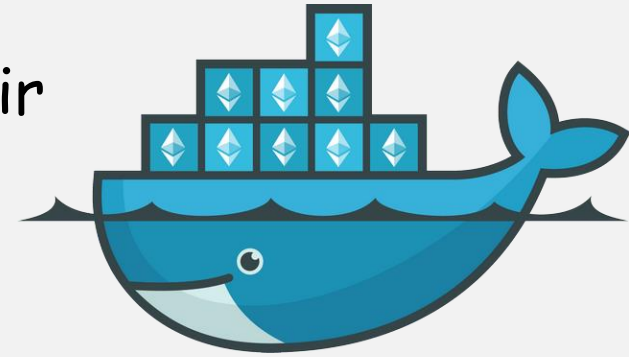
- ✓ Baixa a imagem clojure:tools-deps
- ✓ Cria sua imagem local clj-repl



```
Windows PowerShell
PS D:\clojure> docker build -t clj-repl .
[+] Building 0.2s (6/6) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile            0.0s
=> => transferring dockerfile: 76B                             0.0s
=> [internal] load metadata for docker.io/library/clojure:t    0.0s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                   0.0s
=> [1/2] FROM docker.io/library/clojure:tools-deps@sha256:4    0.0s
=> => resolve docker.io/library/clojure:tools-deps@sha256:4   0.0s
=> CACHED [2/2] WORKDIR /app                                   0.0s
```

`docker run -it clojure:tools-deps clj`

- ✓ Cria e inicia um container novo a partir de uma imagem;
- ✓ A imagem aqui é: `clojure:tools-deps`.



```
Windows PowerShell
PS D:\clojure> docker run -it clojure:tools-deps clj

Clojure 1.12.4
user=> (+ 2 3)
5
user=> |
```

Usando REPL

- ✓ REPL significa **READ EVAL PRINT LOOP** e representa um excelente recurso para iniciar o estudo de **Clojure**;
- ✓ É uma **interface** de **comandos** que permite a avaliação direta de código **Clojure**;
- ✓ No prompt do **REPL**, a primeira linha representa a **versão** do **Clojure**, o qual em nosso caso é a **1.12.4**;



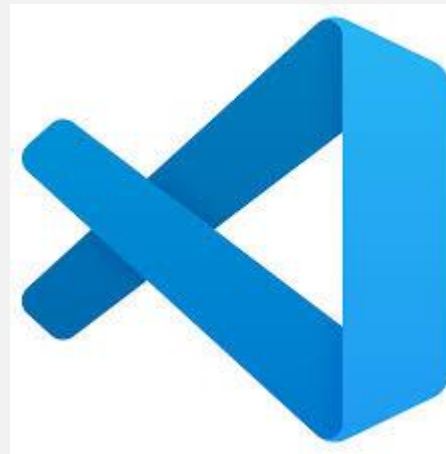
```
Windows PowerShell
PS D:\clojure> docker run -it clojure:tools-deps clj

Clojure 1.12.4
user=> (+ 2 3)
5
user=> |
```



Recomendação

- ✓ Para se aprender **Clojure**, recomenda-se iniciar os estudos com **REPL**;
- ✓ **REPL** permite que se consolide conceitos de **Clojure**, tais como: imutabilidade, estruturas de dados, avaliação de expressões, etc;
- ✓ Num segundo momento, pode-se empregar IDE's como **VS Code** (plugin **Calva**).



Plugin – Atom

 ATOM

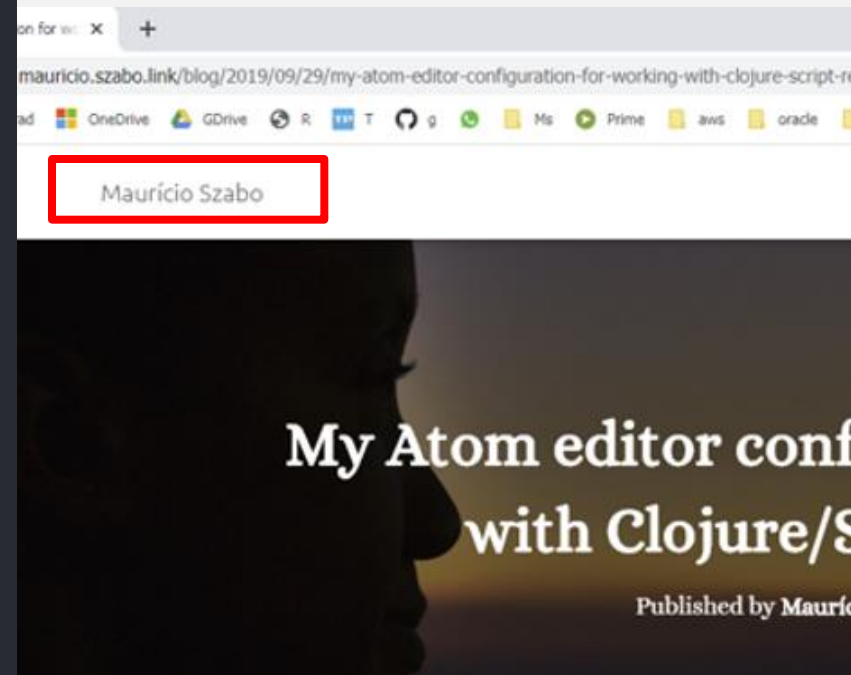
A hackable text editor for the 21st Century

For help, please visit

- The [Atom docs](#) for Guides and the API reference.
- The Atom forum at [discuss.atom.io](#)
- The [Atom org](#). This is where all GitHub-created Atom packages can be found.

☒ Show Welcome Guide when opening Atom

atom.io × 



on for vi x +

mauricio.szabo.link/blog/2019/09/29/my-atom-editor-configuration-for-working-with-clojure-script-re

ad OneDrive GDrive R T g Ms Prime aws orade

Maurício Szabo

My Atom editor conf
with Clojure/S

Published by Maurício

Código Clojure

- ✓ Clojure é um dialeto LISP;
- ✓ LISP significa **List** **P**rocessing;
- ✓ Todo programa **Lisp** é uma **lista** !!!!

```
Clojure 1.12.4  
user=> ( + 2 3 )  
5  
user=> |
```

Clojure – Vantagens



1. Imutabilidade por padrão

- Dados não são alterados, apenas transformados
 - Reduz bugs e facilita o raciocínio
 - Ideal para programação concorrente
-



2. Excelente para concorrência

- Suporte nativo a concorrência (STM, atoms, agents)
- Evita problemas clássicos como race conditions
- Mais seguro que abordagens tradicionais com threads

Clojure – Vantagens

3. Linguagem funcional pura

- Funções são cidadãos de primeira classe
- Incentiva código mais simples, reutilizável e testável
- Menos efeitos colaterais

4. REPL poderoso (desenvolvimento interativo)

- Permite testar código em tempo real
- Aumenta muito a produtividade
- Ideal para aprendizado e exploração

Clojure – Vantagens

5. Extensível com macros

- Permite criar novas estruturas da linguagem
- Alto nível de abstração
- Código mais expressivo

6. Integração total com Java (JVM)

- Usa qualquer biblioteca Java
- Roda na JVM (robusta e madura)
- Fácil integração com sistemas existentes

Clojure – Vantagens



8. Estruturas de dados persistentes

- Estruturas imutáveis eficientes
- Compartilhamento de memória (performance)
- Segurança sem custo alto



9. Boa para APIs e sistemas distribuídos

- Muito usada em backends robustos
- Excelente para microsserviços
- Código previsível e escalável

Clojure – Desvantagens



1. Curva de aprendizado elevada

- Sintaxe baseada em Lisp (muitos parênteses)
- Paradigma funcional pode ser difícil no início
- Diferente de linguagens tradicionais (Java, Kotlin)



2. Mudança de mentalidade

- Exige pensar de forma funcional (imutabilidade, recursão)
- Menos uso de loops e variáveis mutáveis
- Pode gerar estranhamento para iniciantes



3. Debug mais difícil

- Stack traces da JVM podem ser confusos
- Uso de macros pode dificultar entendimento do fluxo
- Menos ferramentas visuais comparado a Java/Kotlin

Clojure – Desvantagens

4. Ecossistema menor

- Menos bibliotecas específicas que Java
- Comunidade menor
- Menos exemplos prontos na internet

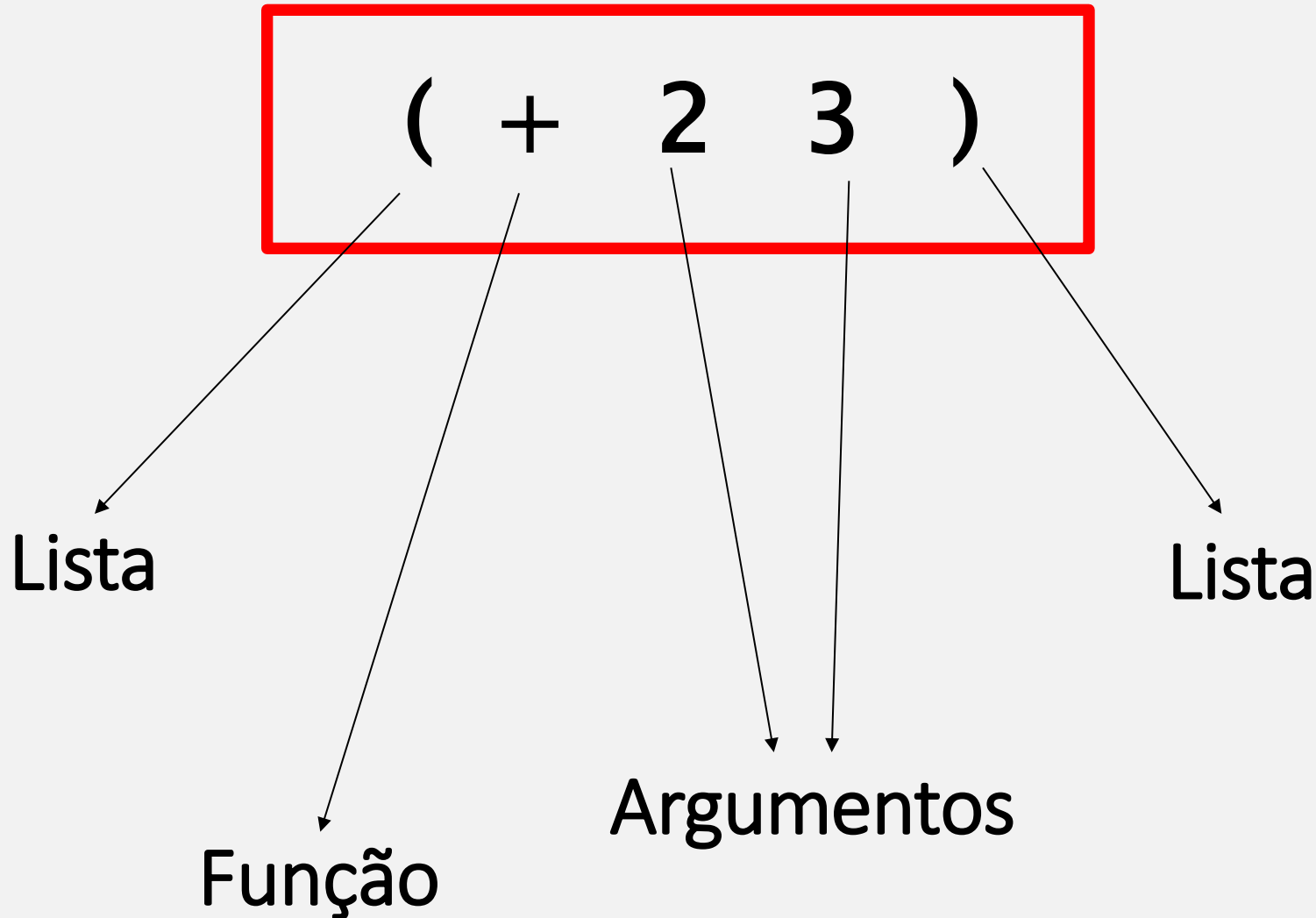
5. Performance inicial pode enganar

- Pode ser mais lenta que Java em alguns cenários
- Requer conhecimento para otimizar (lazy seqs, etc.)
- Overhead da abstração funcional

6. Mercado mais restrito

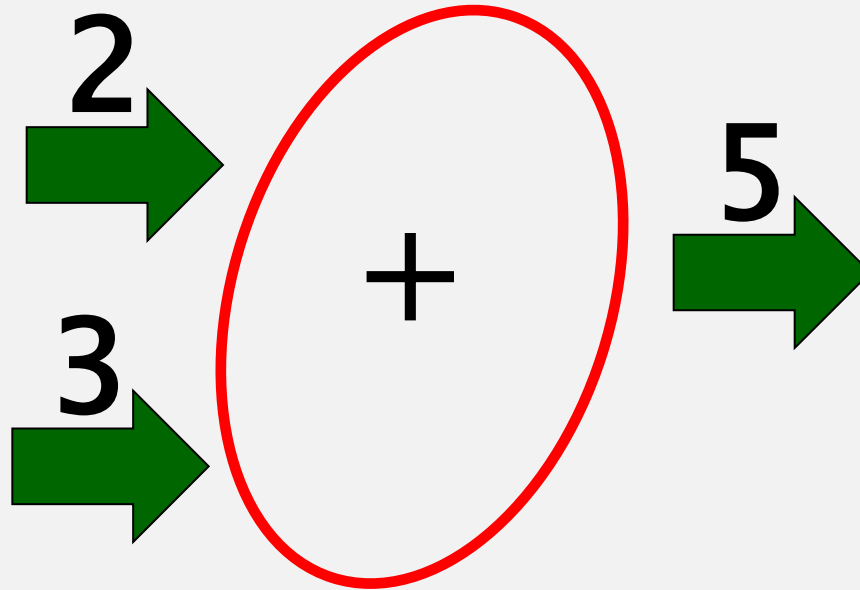
- Menos vagas comparado a Java/Kotlin/JavaScript
- Mais comum em nichos específicos (fintechs, startups)

Código Closure



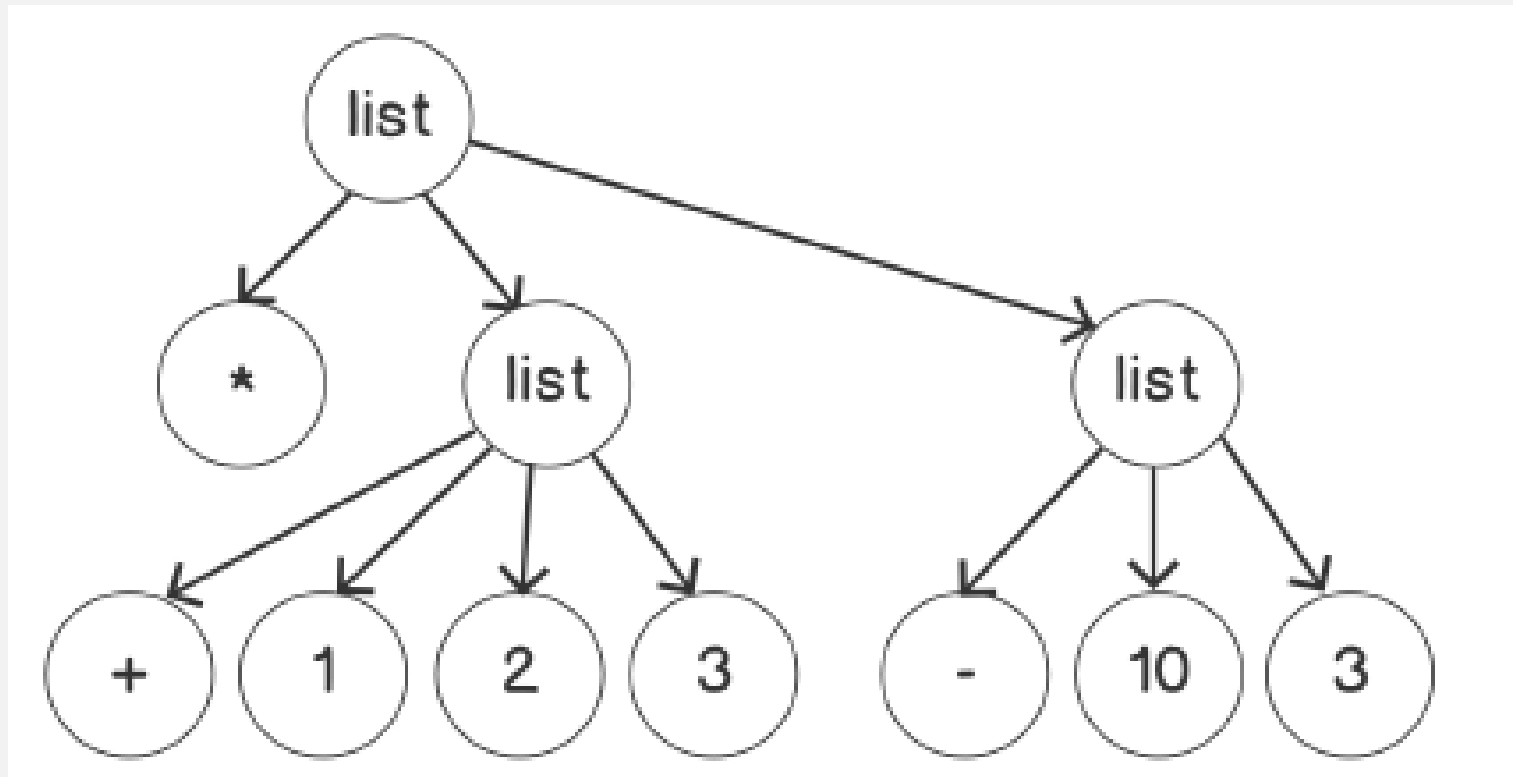
Código Clojure

(+ 2 3)



Expressões podem ser aninhadas

$(* (+ 1 2 3) (- 10 3))$

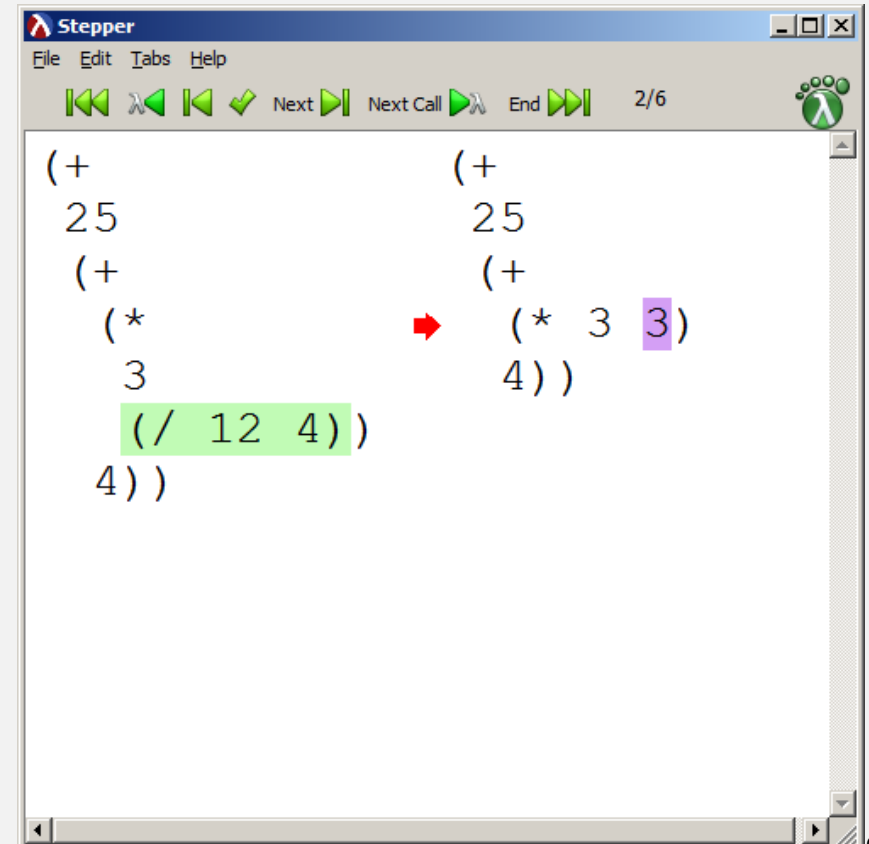
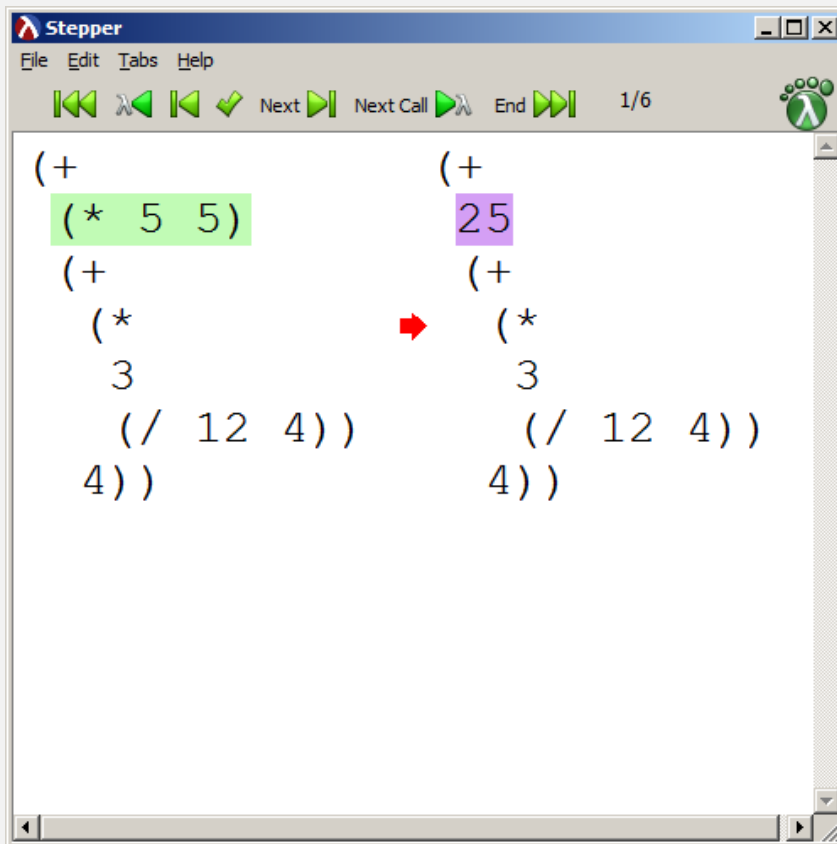
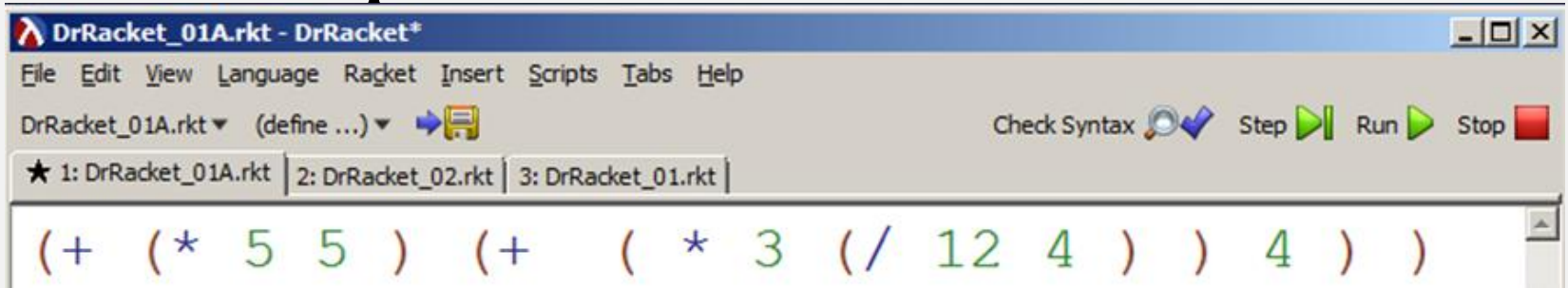


Expressões podem ser aninhadas

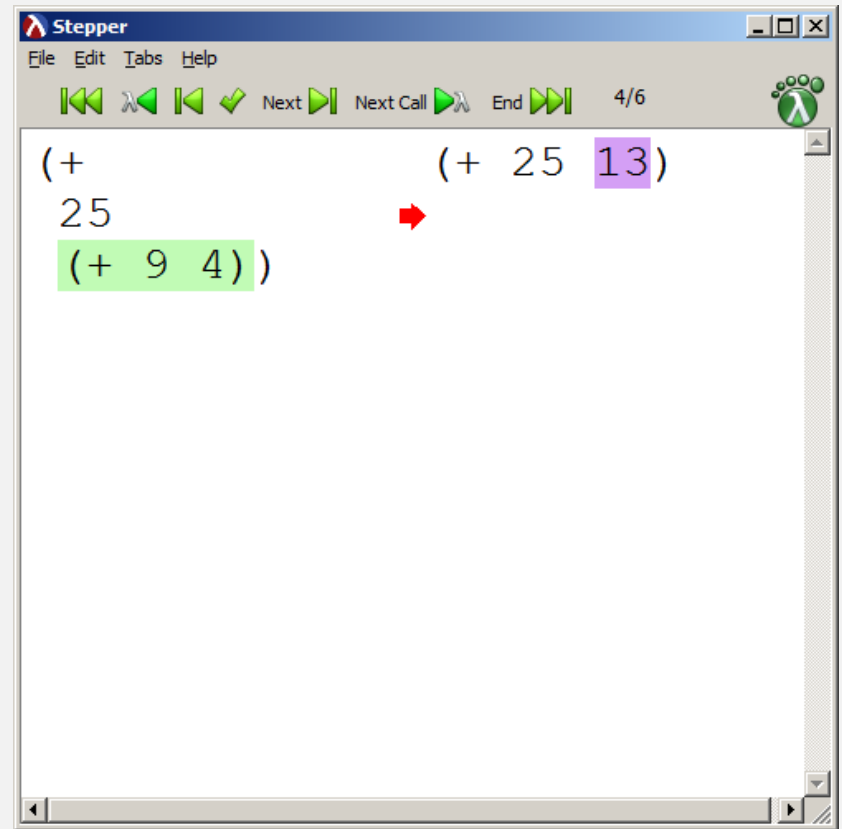
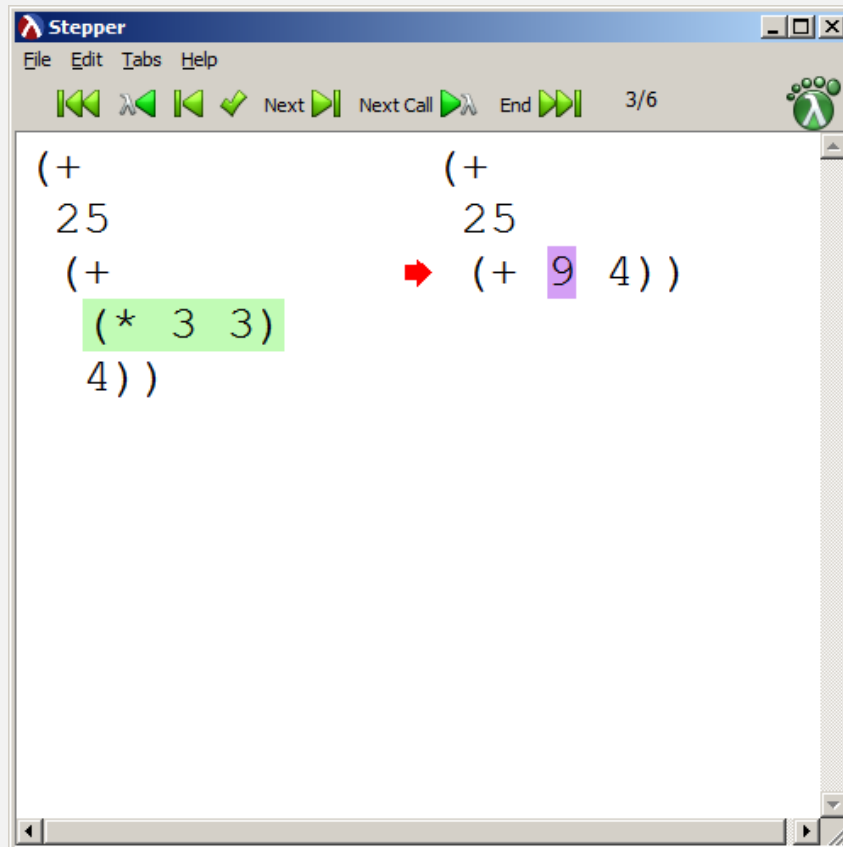
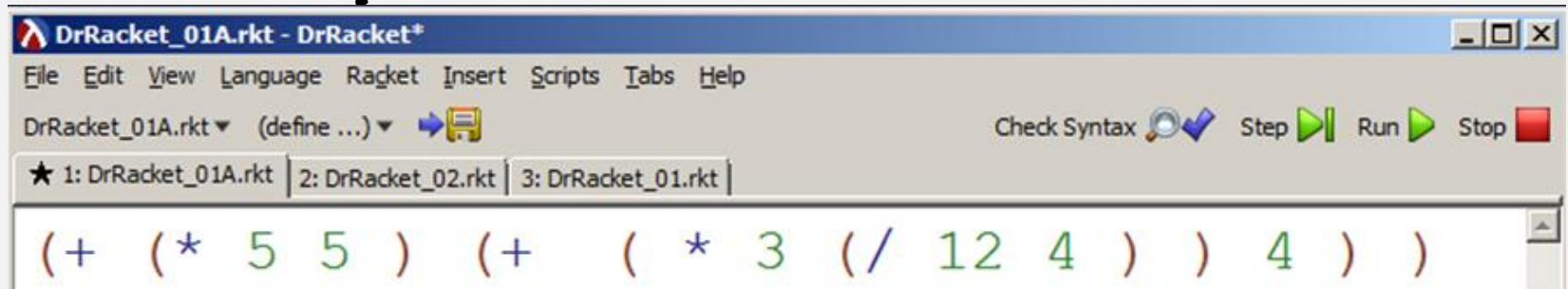
$(* (+ 1 2 3) (- 10 3))$

```
Windows PowerShell
user=>
user=>
user=>
user=>
user=>
user=> ( * ( + 1 2 3 ) ( - 10 3 ) )
42
user=>
```

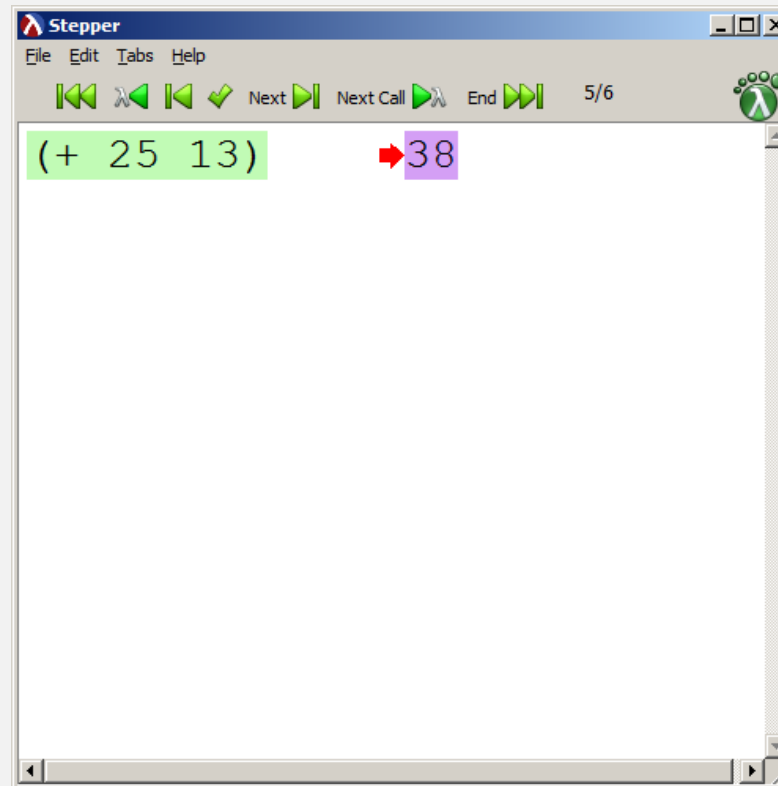
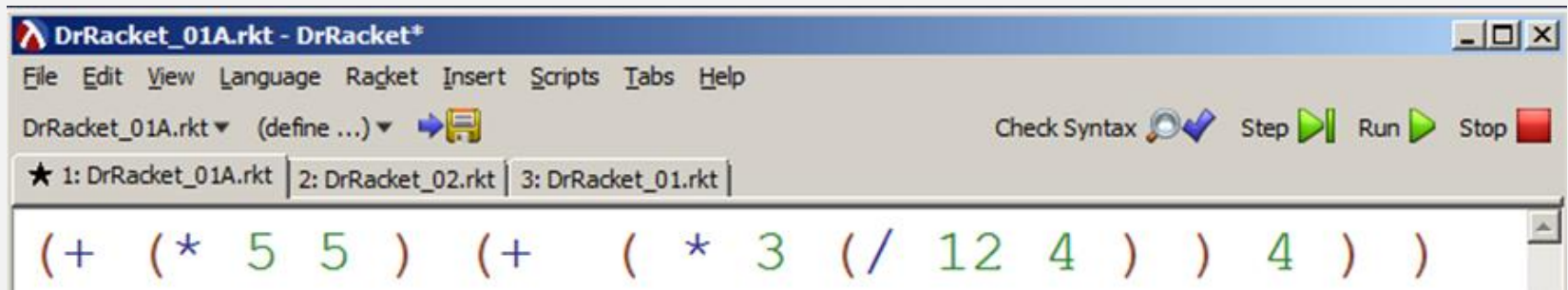
Depuração em Racket



Depuração em Racket



Depuração em Racket



Código Clojure

```
user=>
```

```
user=> (+ (* 5 5) (+ (* 3 (/ 12 4)) 4))
```

```
38
```

```
user=>
```

```
user=>
```

Hello World em C

```
#include <stdio.h>

/* declaração da função */
void hello();

/* função principal */
int main() {
    hello(); // chamada da função
    return 0;
}

/* definição da função */
void hello() {
    printf("Hello World\n");
}
```

Clojure – Hello World

```
user=>  
user=> (defn hello []  
  "Hello World")
```

```
user=>  
user=> (hello)  
"Hello World"  
user=>
```

Hello World

C

```
hello(); // chamada da função
```



Clojure

```
(hello)
```



```
#include <stdio.h>
#include <math.h>

/* função que calcula as raízes */
int raizes_2grau(double a, double b, double c,
                double *x1, double *x2) {

    double delta = b*b - 4*a*c;

    if (delta < 0) {
        return 0;    // nenhuma raiz real
    }

    if (delta == 0) {
        *x1 = -b / (2*a);
        return 1;    // uma raiz real
    }

    *x1 = (-b + sqrt(delta)) / (2*a);
    *x2 = (-b - sqrt(delta)) / (2*a);
    return 2;        // duas raízes reais
}
```

```
(defn raizes-2grau [a b c]
  (if (< (- (* b b) (* 4 a c)) 0)
    nil
    (if (= (- (* b b) (* 4 a c)) 0)
      [(/ (- b) (* 2 a))]
      [(/ (+ (- b) (Math/sqrt (- (* b b) (* 4 a c)))) (* 2 a))
       (/ (- (- b) (Math/sqrt (- (* b b) (* 4 a c)))) (* 2 a))])))
```



```
(defn delta [a b c]
  (- (* b b) (* 4 a c)))

(defn raizes-2grau [a b c]
  (if (< (delta a b c) 0)
    nil
    (if (= (delta a b c) 0)
      [(/ (- b) (* 2 a))]
      [(/ (+ (- b) (Math/sqrt (delta a b c))) (* 2 a))
       (/ (- (- b) (Math/sqrt (delta a b c))) (* 2 a))])))
```



Usando Java em Clojure

- ✓ **Clojure** é hosted language. Isso significa que **Clojure** usa **JVM** ao invés de criar um novo ambiente de runtime;
- ✓ Assim, **Clojure** reusa todas as facilidades providas pela **JVM**;
- ✓ Assim, todo desenvolvedor **Clojure** tem acesso ao ecossistema de bibliotecas **JVM**;



Ferramentas Backend



clj-pkg/jwt

reitit

roteador HTTP



Apache Tomcat

seancorfield/**next-jdbc**

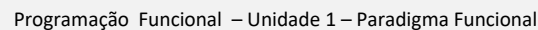
A modern low-level Clojure wrapper for JDBC-based access to databases.



dakrone/**cheshire**

Clojure JSON and JSON SMILE (binary json format) encoding/decoding

- ✓ A maioria das linguagens de Programação tiveram suas origens no **Fortran**.



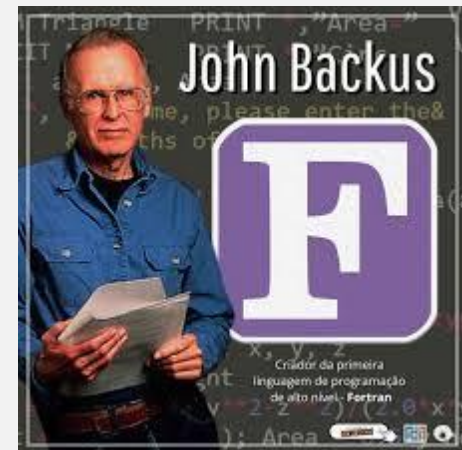
O Paradigma Funcional

- ✓ **Fortran** foi a primeira linguagem de programação de alto nível amplamente utilizada.
- ✓ O primeiro compilador **Fortran** foi desenvolvido por uma equipe da **IBM** sendo chefiada por **John Backus**, na década de 50;

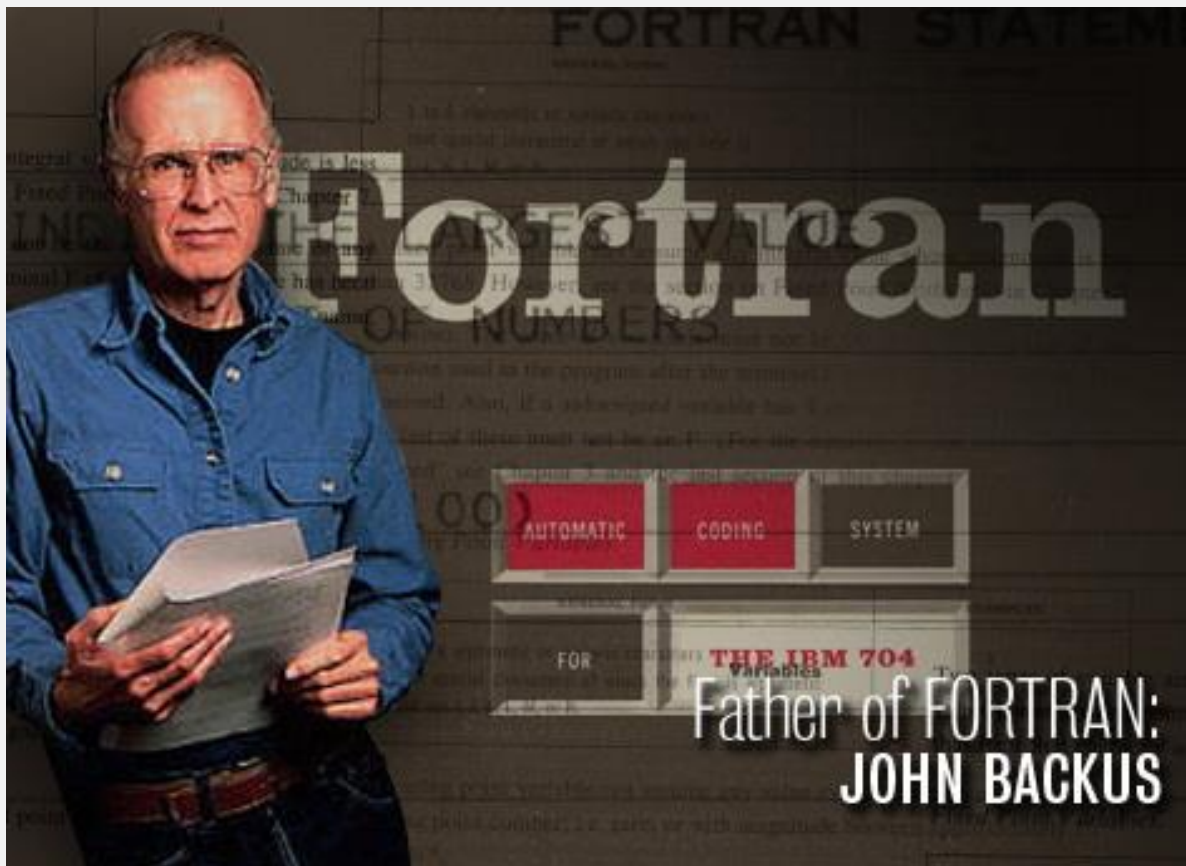


O Paradigma Funcional

- ✓ Fortran é uma linguagem fortemente **aderente** ao paradigma **imperativo** e seu projeto tinha como grande objetivo o uso **eficiente** de máquina;
- ✓ Em 1977 na palestra ministrada por John Backus quando ganhou o prêmio ACM Turing, ele argumentou que **linguagens funcionais** são melhores que as **imperativas**, pois podem apresentar mais **confiabilidade**, **legibilidade** e maior probabilidade de estarem **corretas**.

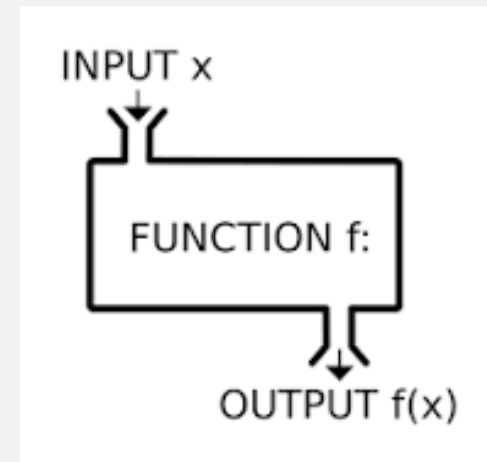


Em que se baseou Backus para afirmar que Linguagens Funcionais apresentam maior legibilidade e confiabilidade ?



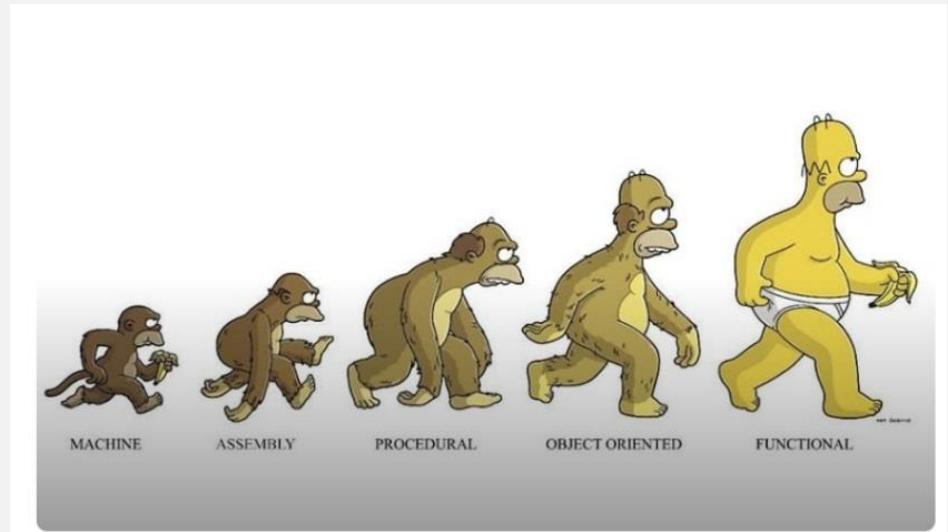
Linguagens Funcionais

- ✓ O argumento de **John Backus** teve como base o seguinte conceito: Em Linguagens Puramente Funcionais, funções apresentam transparência referencial: a mesma entrada sempre produz a mesma saída.
- ✓ Em **Linguagens Puramente Funcionais** nem expressões nem funções apresentam Efeitos Colaterais (**Side Effects**);
- ✓ **Backus** propôs na época uma nova Linguagem Funcional chamada **FP** (Functional Programming) para embasar seu argumento;
- ✓ A linguagem não vingou, mas abriu espaço para pesquisa do Paradigma Funcional de Programação;



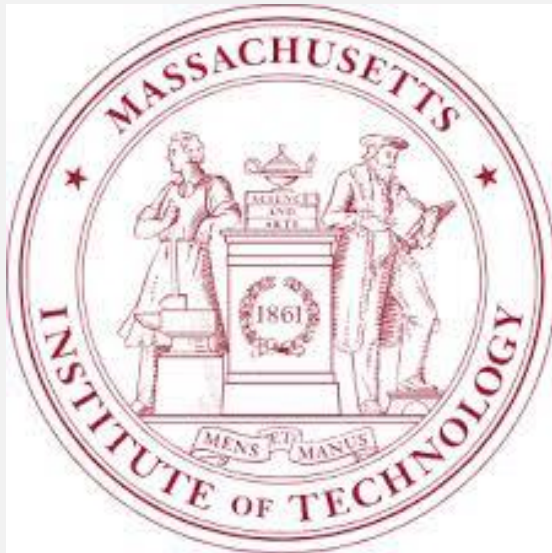
Programas Funcionais x Imperativos

- ✓ Uma das principais características dos programas escritos em Linguagens Imperativas é que eles possuem **ESTADO**;
- ✓ Ou seja, a execução do programa corresponde à sucessivas mudanças de estado em variáveis, no qual a resposta do programa será representada pelos estados finais de suas **variáveis** (Transformação de Estado);
- ✓ Para grandes programas, esta tarefa pode ser difícil;
- ✓ Em programas escritos com **Linguagens Puramente Funcionais** estes problemas não ocorrem, pois programas funcionais evitam estado mutável e utilizam variáveis imutáveis.



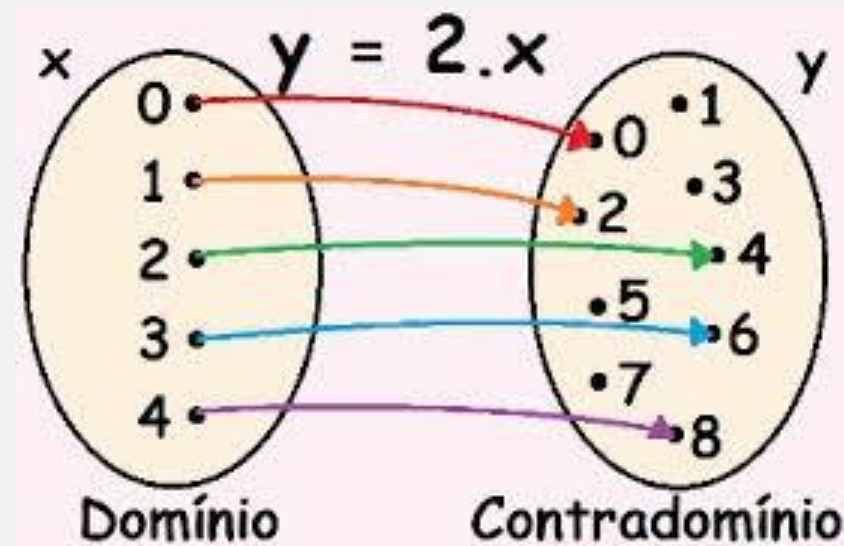
Linguagem de Programação Lisp

- ✓ Concebida por John McCarthy em 1959, no **MIT**;
- ✓ Focada no uso exclusivo de funções matemáticas como estrutura de dados;
- ✓ Tem como base formal o **Cálculo Lambda** de **Alonzo Church**;
- ✓ Uma das primeiras e mais influentes linguagens do paradigma funcional.



Funções Matemáticas

- ✓ Uma **função** matemática é um **mapeamento** de elementos de um conjunto, chamado conjunto **Domínio**, para outro conjunto, chamado **Contradomínio** (range set);
- ✓ O **mapeamento** é descrito por uma expressão;
- ✓ **Funções** são aplicadas a um elemento específico do **Domínio**, passado como argumento para a função, obtendo-se um valor do **Contradomínio**.



Paradigma Imperativo

- ✓ O programa é descrito como uma sequência de comandos que alteram o estado do sistema.
- ✓ O foco está em **como** o problema será resolvido.
- ✓ As variáveis podem ser **modificadas** ao longo da execução.
- ✓ Funções podem produzir efeitos colaterais (side effects), como:
 - ❖ alterar variáveis globais
 - ❖ modificar estruturas de dados
- ✓  Exemplos de linguagens: C, Java, Python.



Paradigma Funcional

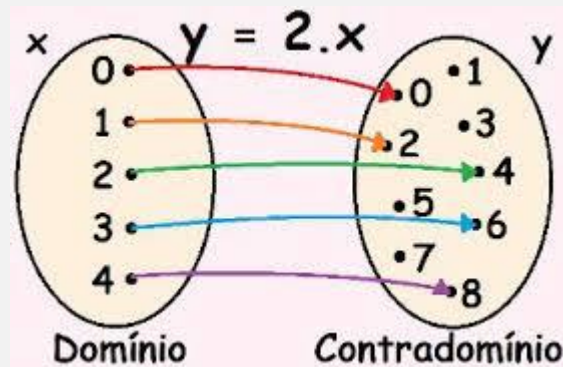
- ✓ O programa é construído a partir da composição e avaliação de funções matemáticas.
- ✓ As funções são determinísticas: a mesma entrada sempre produz a mesma saída.
- ✓ Em linguagens puramente funcionais **não** existem efeitos colaterais.
- ✓ O significado das expressões é independente do contexto.

📌 Exemplos de linguagens: Haskell, Clojure, Lisp.



Funções Matemáticas

- ✓ Na Matemática, não existe algo como uma variável que é usada para modelar uma localização de memória;
- ✓ Variáveis locais nas linguagens imperativas mantêm o **estado** da função. Nessas linguagens, a computação é realizada pela avaliação das expressões que **modificam** o **estado** do programa, por meio de **comandos** de atribuição;
- ✓ Na Matemática, por outro lado, não existe o conceito de **estado de uma função**;
- ✓ Uma função matemática **sempre** mapeia seu parâmetro (ou parâmetros) para um valor (ou valores) ao invés de se especificar uma sequência de operações em valores de memória para computar um resultado.



Funções Simples

- ✓ Definições de funções são frequentemente escritas na forma: nome da função, seguido por uma lista de parâmetros entre parênteses e uma **expressão** que define o mapeamento;
- ✓ Por exemplo:

$\text{cubo}(x) \equiv x * x * x$, onde x é um número real.

- ✓ Nessa função, o Domínio e o Contradomínio são o conjunto dos reais;
- ✓ O símbolo $\equiv$ é usado para "é definida como";
- ✓ O parâmetro x representa qualquer elemento $\in$ ao domínio, mas assume um **valor específico** durante a **avaliação** da expressão;
- ✓ Em programação funcional, uma função deve se comportar como uma função matemática.

Aplicação de Funções

- ✓ A aplicação de uma função consiste em associar o nome da função a um elemento específico do domínio.
- ✓ O elemento do Contradomínio é obtido pela **avaliação** da **expressão** de mapeamento com o valor do domínio substituído pelas ocorrências do parâmetro;
- ✓ É importante observar que durante a avaliação, **toda ocorrência de um argumento é ligada (bound)** a um **valor** do domínio, sendo **constante** durante toda a avaliação;
- ✓ Exemplo:

$$\text{cubo}(x) \equiv x * x * x$$

Definição

$$\text{cubo}(2.0) \equiv 2.0 * 2.0 * 2.0 = 8$$



Aplicação

Aplicação de Funções

- ✓ O argumento x é ligado (bound) ao valor **2.0** durante a avaliação;
- ✓ Não há argumentos **sem ligação** (unbound);
- ✓ Adicionalmente, x é uma constante (seu valor não pode ser alterado) **durante a avaliação**.
- ✓ Assim, o argumento passado à função é **imutável**!
- ✓ Esse conceito é chamado **Imutabilidade**.

$\text{cubo}(x) \equiv x * x * x$ Definição

$\text{cubo}(2.0) \equiv 2.0 * 2.0 * 2.0 = 8$



Aplicação

Exemplo – Clojure

```
(defn cubo [x]  
  (* x x x))
```

Aplicação:

```
(cubo 2)
```

Resultado:

8

- ✓ O parâmetro `x` é ligado (bound) ao argumento `2`.
- ✓ Durante a avaliação da função, todas as ocorrências de `x` representam o mesmo valor.
- ✓ O valor de `x` não pode ser alterado.

Exemplo – Clojure

Definição da função:

⟨/⟩ clojure

```
(defn dobro [x]  
  (* x 2))
```

Aplicação da função:

⟨/⟩ clojure

```
(dobro 5)
```

Exemplo – Clojure

Processo de Avaliação

A avaliação ocorre por substituição do parâmetro pelo argumento.

```
(defn dobro [x]  
  (* x 2))
```

Na programação funcional:

- ✓ funções se comportam como **funções matemáticas**
- ✓ parâmetros são **imutáveis**
- ✓ avaliação ocorre por **substituição e redução da expressão**

Passo 1

```
</> clojure
```

```
(dobro 5)
```

Passo 2 — substituição do parâmetro

```
</> clojure
```

```
(* 5 2)
```

Passo 3 — avaliação da expressão

```
10
```

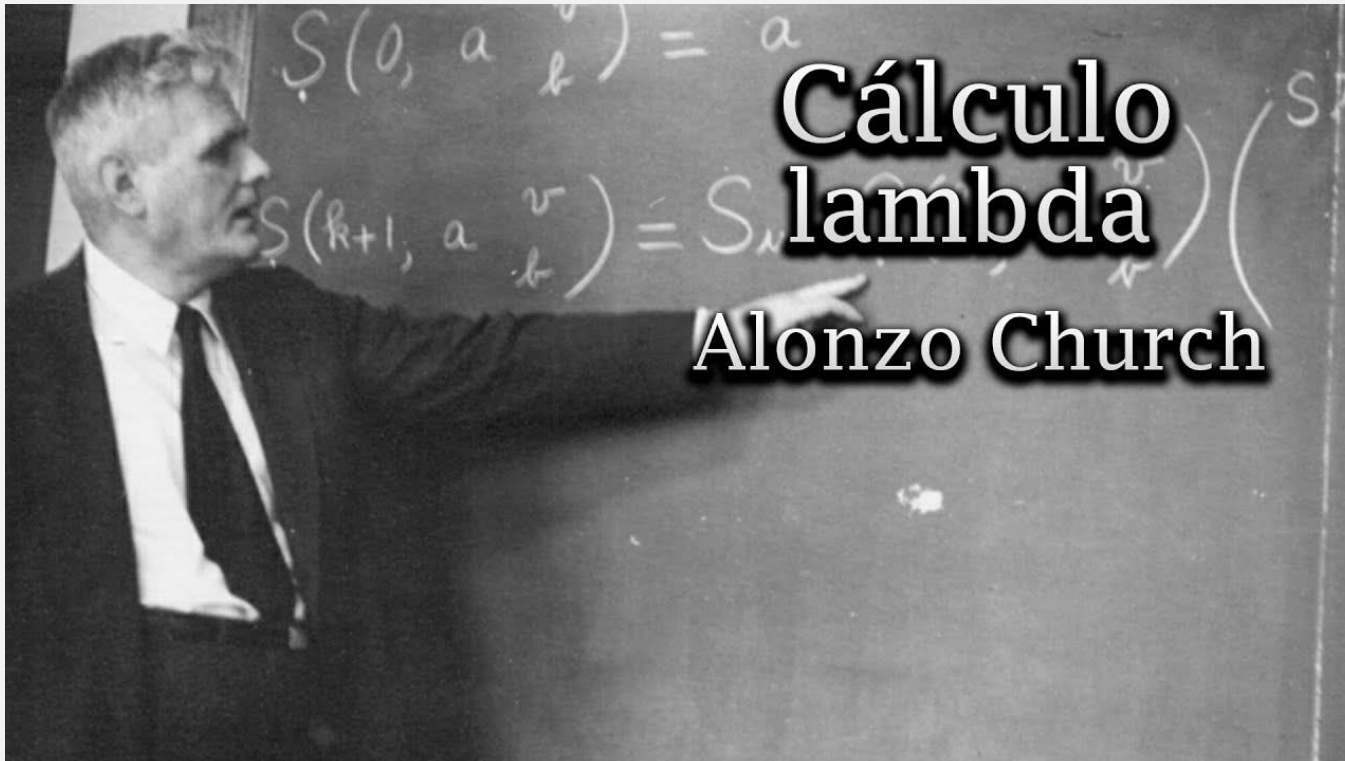
Cálculo Lambda

- ✓ A **base teórica** do Paradigma Funcional está no **Cálculo Lambda**;
- ✓ Nessa teoria, funções são definidas **sem nome** (**Notação Lambda**);
- ✓ Uma expressão **Lambda** especifica os parâmetros e o mapeamento de uma função;
- ✓ A **expressão lambda**, portanto, é a própria função (sem nome);
- ✓ Por exemplo:

$$\lambda(x) \ x * x * x$$


Cálculo Lambda

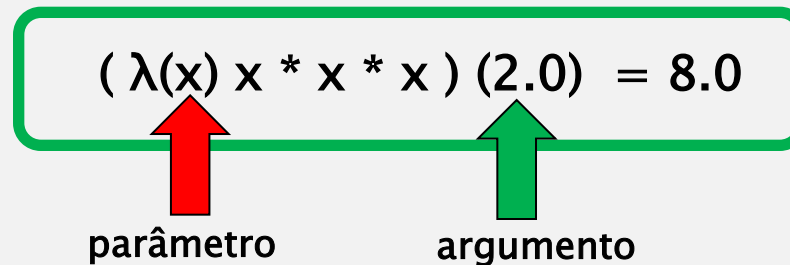
- ✓ Church (1941) definiu um modelo formal de computação (um sistema formal para definição de funções, aplicação de funções e recursão) com o emprego de expressões lambda;
- ✓ As linguagens puramente funcionais baseiam-se no **Cálculo Lambda**;



Notação Lambda para Aplicação de Funções

- ✓ Durante a avaliação, o parâmetro é ligado (bound) a um valor específico (argumento) que permanece **constante** durante a avaliação;
- ✓ Quando uma expressão lambda é avaliada para um determinado argumento, diz-se nesse caso, que a expressão é **aplicada** a este **argumento**;
- ✓ O mecanismo usado nesse caso é o mesmo que o ocorre na avaliação de qualquer função;

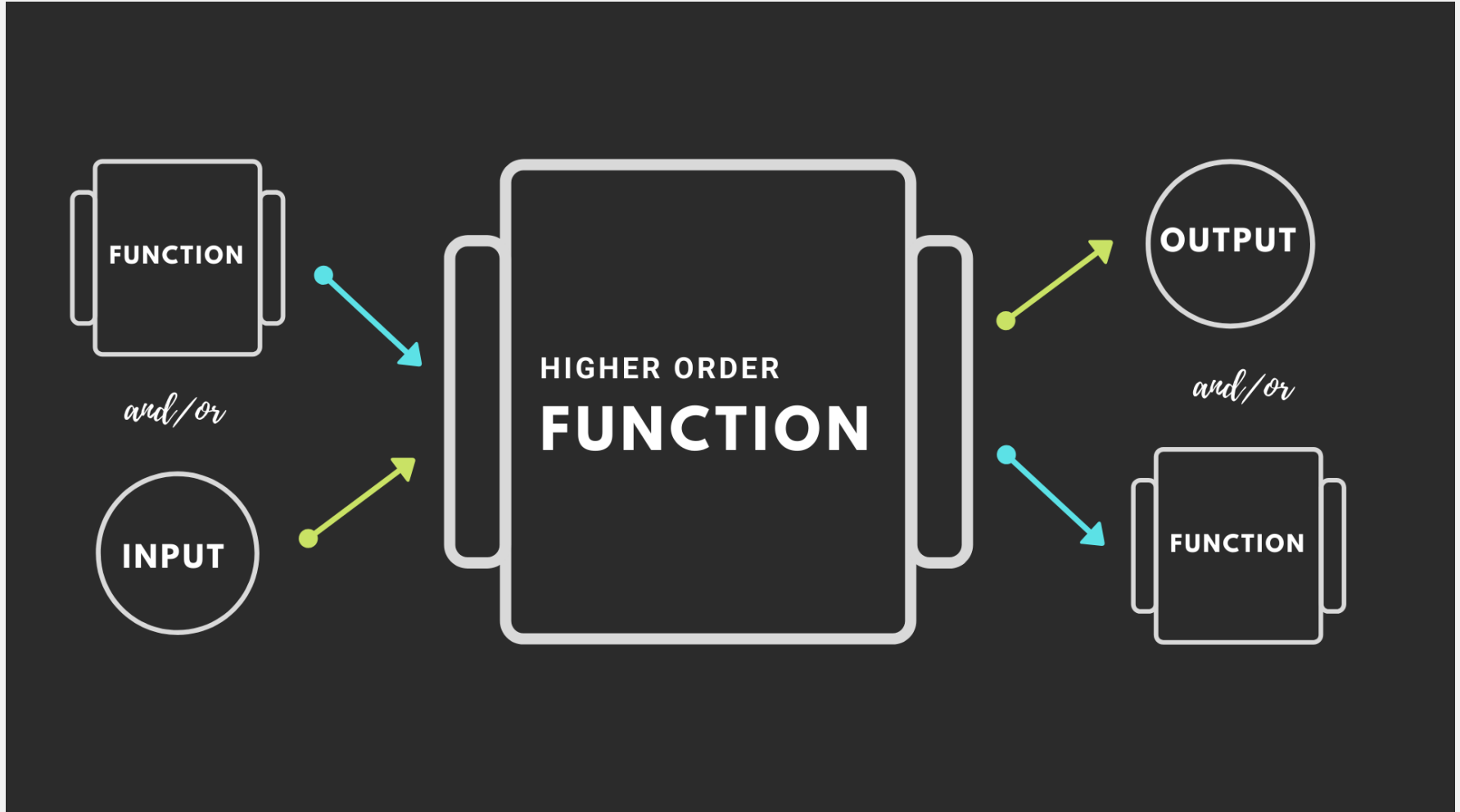
✓ Exemplo:


$$(\lambda(x) x * x * x) (2.0) = 8.0$$

parâmetro argumento

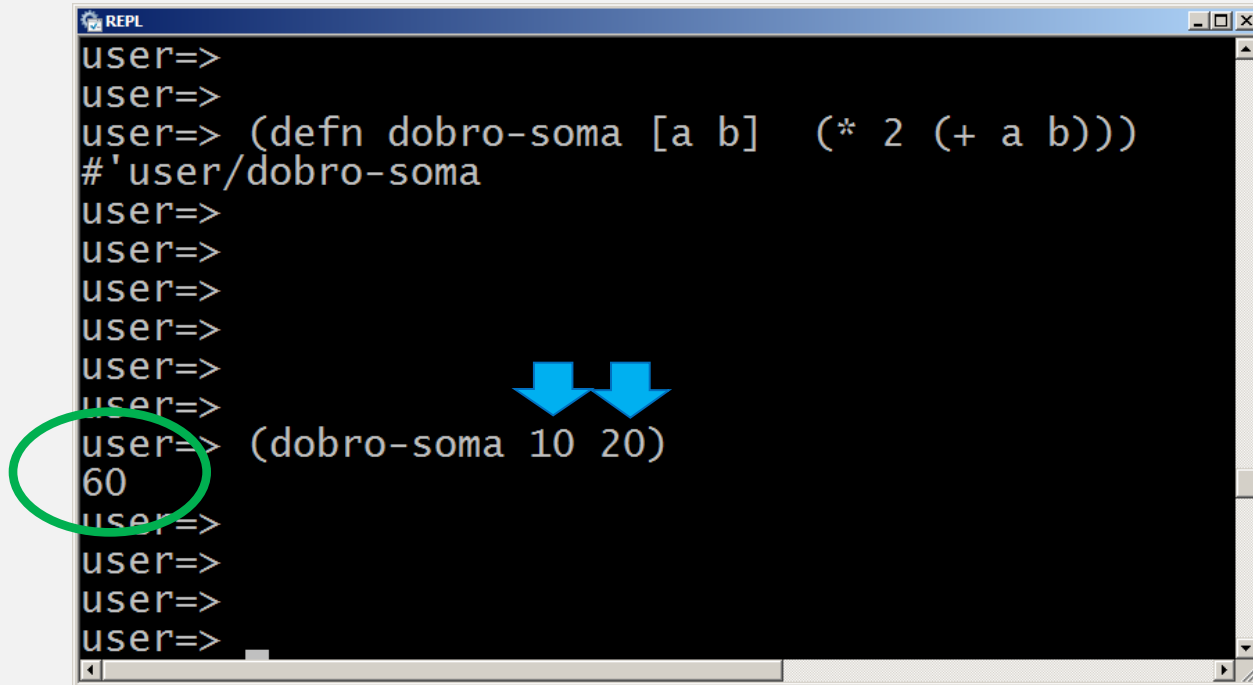
Higher Order Function

- ✓ Funções podem receber funções como parâmetros e também **produzir funções** como resultado.



Higher Order Function – Clojure

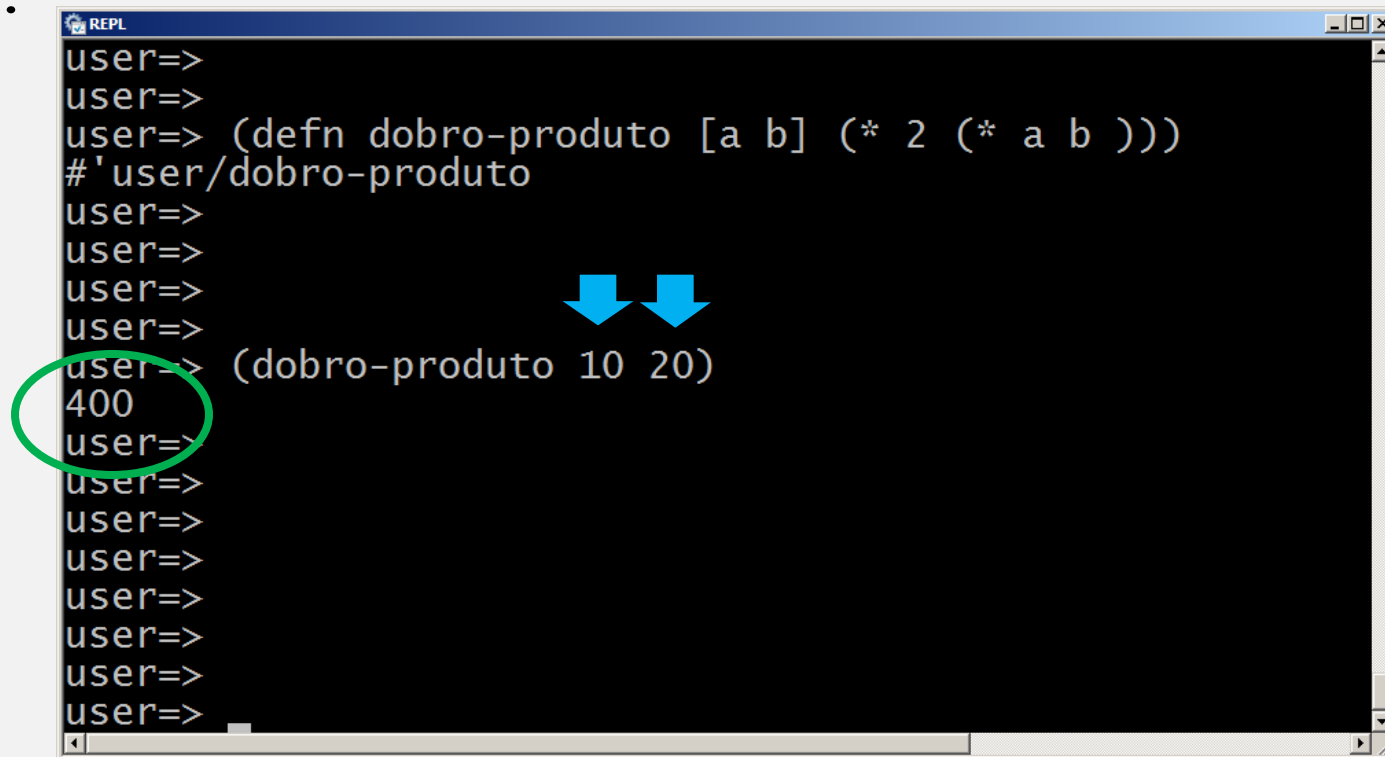
- ✓ A função **dobro-soma** possui 2 parâmetros, **a** e **b**;
- ✓ A função retorna o dobro da soma dos **argumentos a** e **b** passados a ela;
- ✓ Por exemplo, para os argumentos **10** e **20**, a função retorna o valor **60**.



```
REPL
user=>
user=>
user=> (defn dobro-soma [a b] (* 2 (+ a b)))
#'user/dobro-soma
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (dobro-soma 10 20)
60
user=>
user=>
user=>
```

Higher Order Function – Clojure

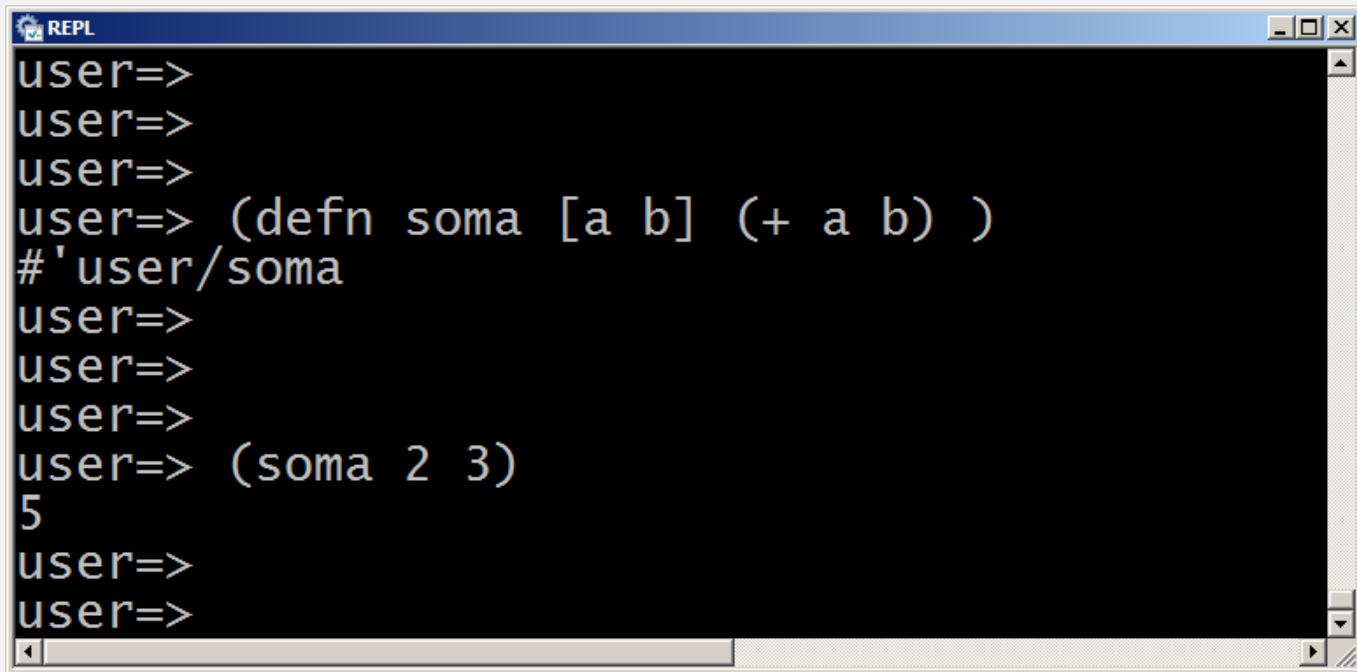
- ✓ A função **dobro-produto** também possui 2 parâmetros, **a** e **b**;
- ✓ A função retorna o dobro do produto dos **argumentos a e b** passados a ela;
- ✓ Por exemplo, para os argumentos **10** e **20**, a função retorna o valor **400**.



```
REPL
user=>
user=>
user=> (defn dobro-produto [a b] (* 2 (* a b )))
#'user/dobro-produto
user=>
user=>
user=>
user=>
user=> (dobro-produto 10 20)
400
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
```

Higher Order Function – Clojure

- ✓ As funções **dobro-soma** e **dobro-produto**, compartilham um padrão;
- ✓ Elas somente diferem no nome na função usada para a computação de a e b;
- ✓ Poderíamos definir uma função chamada **soma** e passá-la para **dobro-soma**.



```
REPL
user=>
user=>
user=>
user=> (defn soma [a b] (+ a b) )
#'user/soma
user=>
user=>
user=>
user=> (soma 2 3)
5
user=>
user=>
```

Higher Order Function – Clojure

✓ Poderíamos definir uma função chamada **soma** e passá-la para **dobro-soma**.

```
REPL
user=>
user=>
user=>
user=> (defn soma [a b] (+ a b) )
#'user/soma
user=>
user=>
user=>
user=> (soma 2 3)
5
user=>
user=>
```

```
REPL
user=>
user=> (defn dobro-soma [soma a b] (* 2 (soma a b)))
#'user/dobro-soma
user=>
user=>
user=>
user=>
user=>
user=>
user=> (dobro-soma soma 10 20)
60
user=>
```

Higher Order Function

✓ A composição matemática pode ser vista como um caso especial de função de alta ordem.

✓ Se: $f(x) \equiv x + 2$

$g(x) \equiv 3 * x$

$$h \equiv f \circ g$$

✓ Então h é definida por: $h(x) \equiv f(g(x))$, ou $h(x) \equiv (3 * x) + 2$

✓ Passa-se para a função $f(x)$ uma outra função $g(x)$:

✓ O parâmetro para $f(x)$ é a função $g(x)$.

Objetivos da Programação Funcional

- ✓ Focar no emprego de funções matemáticas da forma mais intensa possível.
- ✓ Isso resulta numa abordagem totalmente diferente da Programação Imperativa;



Lambda

$f A : (A, A) \rightarrow A$



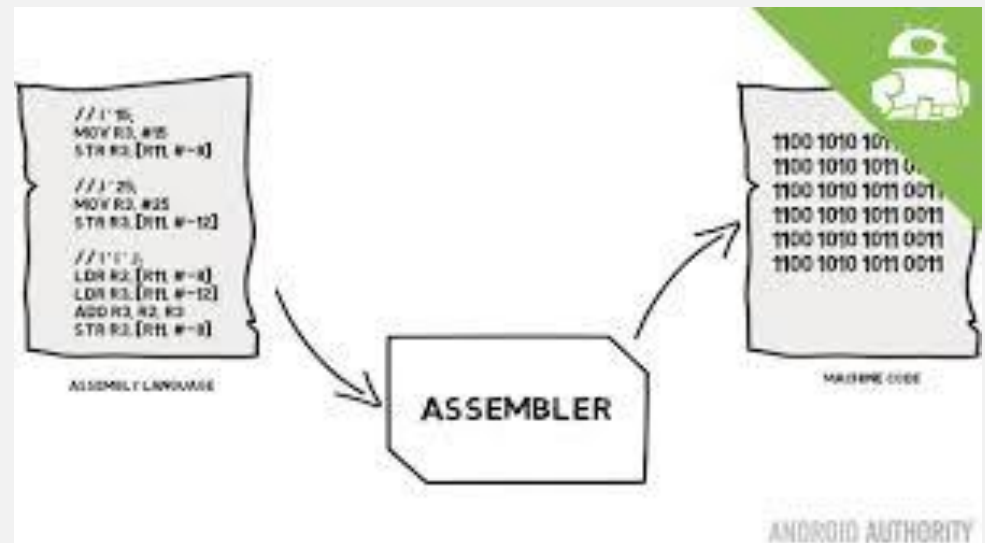
$\text{map } B : (B, B) \rightarrow B$



$\text{flatMap } A : (A, A) \rightarrow A$

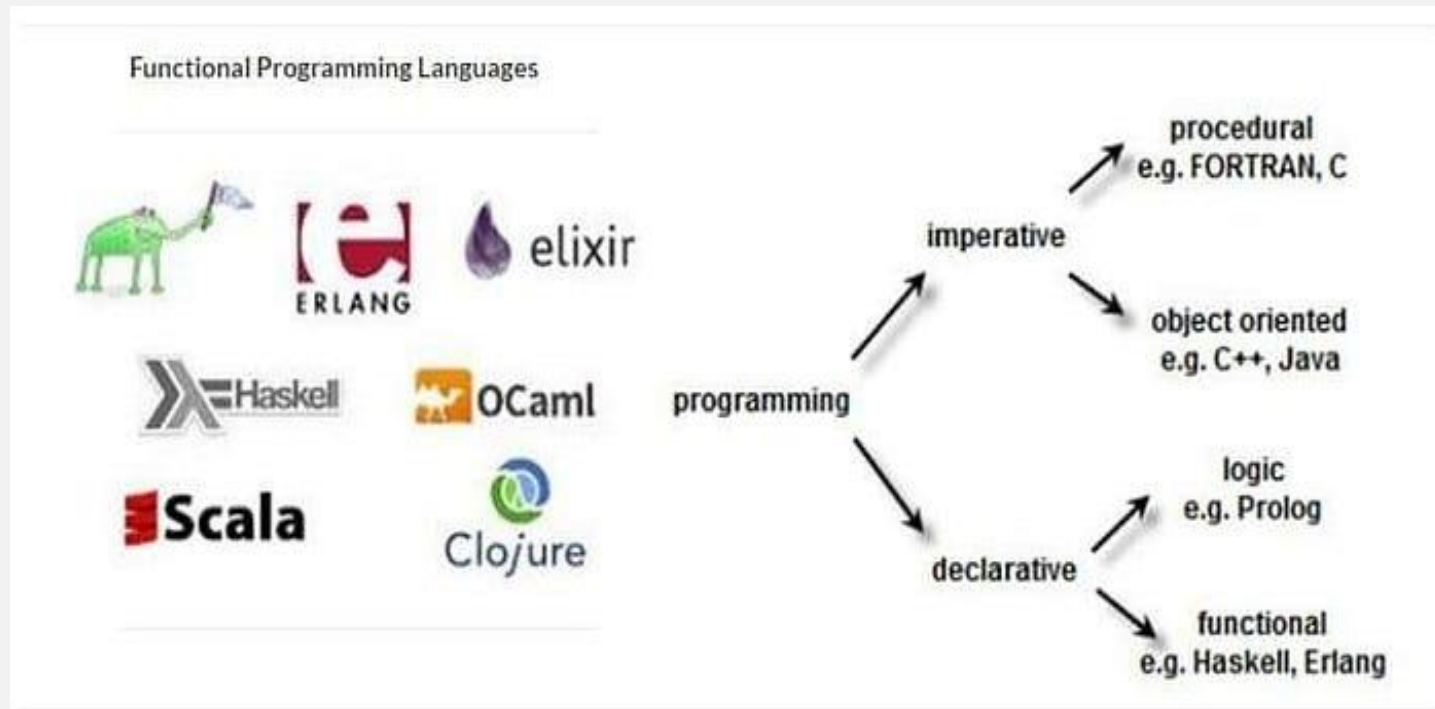
Programação Funcional x Imperativa

- ✓ Na programação imperativa, uma expressão é avaliada e seu resultado é armazenado em alguma localização de memória, a qual é representada por uma variável (computação feita por sequência de comandos de atribuição);
- ✓ Essa abordagem na programação imperativa se concentra no emprego de células de memória, cujos valores representam o **estado** do programa, resultando assim numa metodologia de programação baixo-nível (baixa abstração);



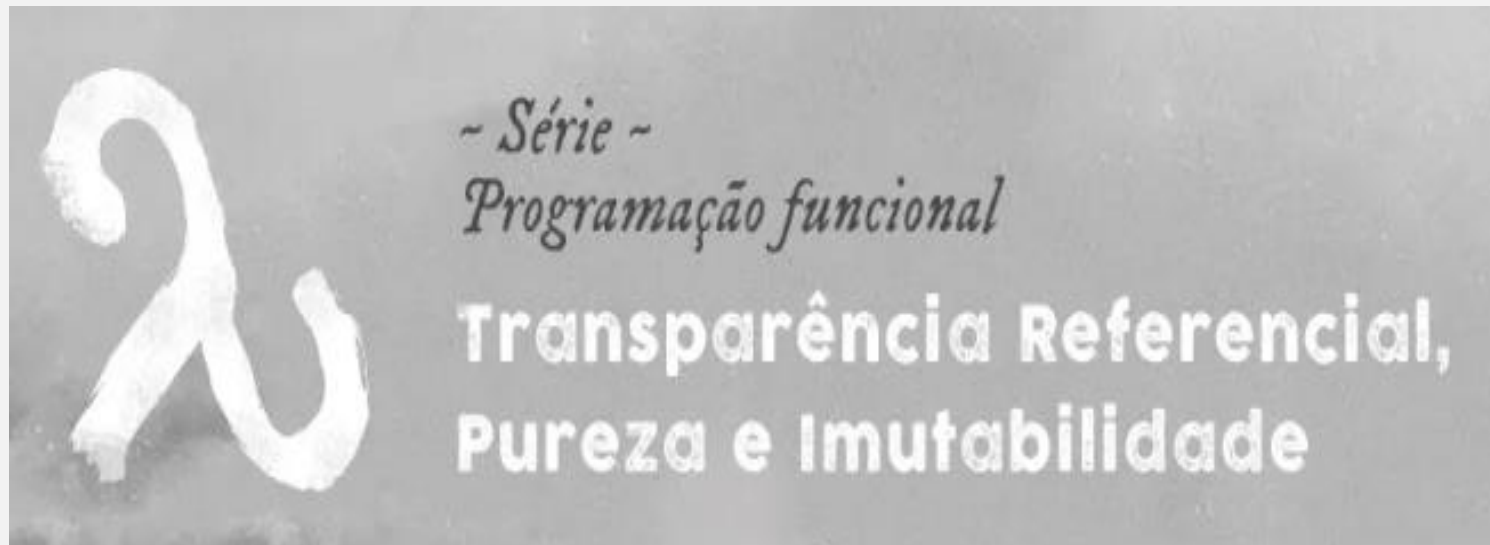
Programação Funcional x Imperativa

- ✓ Uma linguagem de programação funcional não usa variáveis ou statements de atribuição, liberando assim o programador de lidar com células de memória e estados de execução;
- ✓ Repetição é geralmente expressa por recursão em vez de loops baseados em variáveis mutáveis.



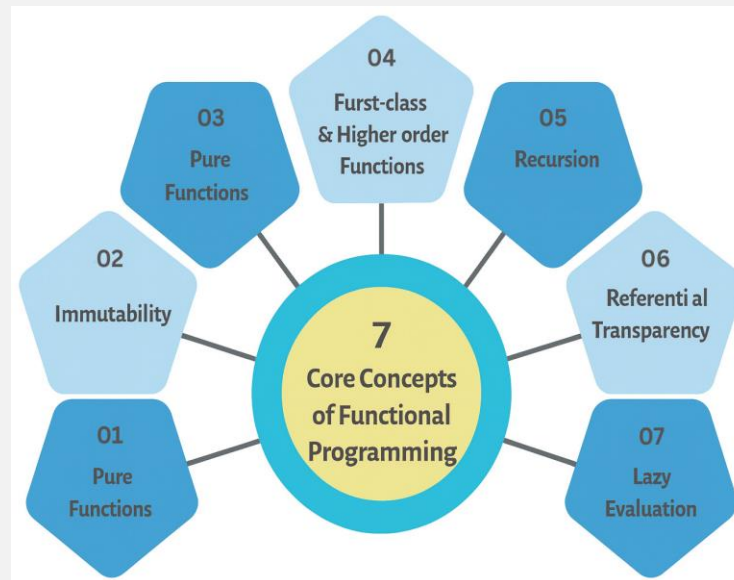
Programação Funcional x Imperativa

- ✓ Programas em linguagens funcionais são **definições de funções e especificação de aplicação de funções**;
- ✓ A execução desses programas corresponde à **avaliação** da aplicação das funções;
- ✓ A execução de uma função **sempre produz o mesmo resultado** para os mesmos parâmetros de entrada (**Transparência Referencial**).



Linguagens Funcionais

- ✓ Provêem um **conjunto de funções primitivas**;
- ✓ Provêem um conjunto de **formas funcionais** (Higher Order Functions) para que funções complexas possam construídas a partir das primitivas;
- ✓ Provêem uma operação de **aplicação** da função; Provêem algumas **estruturas** para representar **dados**. Essas estruturas são usadas para representar parâmetros e valores computados pelas funções.



A Linguagem Lisp

- ✓ Primeira Linguagem do Paradigma Funcional;
- ✓ Desenvolvida no **MIT** em 1959;
- ✓ Atualmente existem diversos **dialetos** do **Lisp** que incluem algumas características das linguagens imperativas;

